

ENGENHARIA E PROJETO DE *SOFTWARE*

SUMÁRIO

UNIDADE 1 - EVOLUÇÃO DO SOFTWARE, FUNDAMENTOS DE ENGENHARIA DE SOFTWARE, CICLO DE VIDA DE DESENVOLVIMENTO DE

Sumário

UNIDADE 1.....	1
OBJETIVOS DE APRENDIZAGEM	1
PLANO DE ESTUDOS.....	1
TÓPICO 1.....	3
1INTRODUÇÃO	3
3TIPOS DE SOFTWARE DO PONTO DE VISTA DA ENGENHARIA.....	9
RESUMO DO TÓPICO 1	14
1INTRODUÇÃO	16
3METODOLOGIA DE DESENVOLVIMENTO DE SISTEMAS.....	20
RESUMO DO TÓPICO 2	31
1INTRODUÇÃO	33
2MODELOS DE PROCESSO DE CICLO DE VIDA DE SOFTWARE.....	34
RESUMO DO TÓPICO 3	Erro! Indicador não definido.
1INTRODUÇÃO	50
2TÉCNICAS DE LEVANTAMENTO DE REQUISITOS	54

UNIDADE 1

EVOLUÇÃO DO SOFTWARE, FUNDAMENTOS DE ENGENHARIA DE SOFTWARE, CICLO DE VIDA DE DESENVOLVIMENTO DE SOFTWARE E REQUISITO DE SOFTWARE

OBJETIVOS DE APRENDIZAGEM

Ao final desta unidade você será capaz de:

- compreender os principais conceitos da evolução do *software*;
- entender a área de Engenharia de *Software* de maneira sistêmica;
- compreender as principais metodologias de ciclo de vida de *software* e processos de *software*;
- conhecer a importância de requisitos de *software*.

PLANO DE ESTUDOS

Esta unidade de ensino contém quatro tópicos. No final de cada um deles você encontrará atividades que contribuirão para a apropriação dos conteúdos.

TÓPICO 1 – EVOLUÇÃO DO SOFTWARE

TÓPICO 2 – FUNDAMENTOS DE ENGENHARIA DE SOFTWARE

TÓPICO 3 – CICLO DE VIDA DE DESENVOLVIMENTO DE SOFTWARE

TÓPICO 4 – REQUISITO DE SOFTWARE

1 INTRODUÇÃO

EVOLUÇÃO DO SOFTWARE

A evolução do *software* confunde-se com a evolução dos computadores, inicialmente compostos apenas do *hardware*, onde toda a lógica de processamento era executada no meio físico. Porém, à medida que o *hardware* evolui, o *software* também acompanha essa mudança devido à necessidade de se tornar acessível ao usuário final.

O computador necessita que o *software* e o *hardware* andem em paralelo, para que haja um melhor aproveitamento dos recursos. De nada adianta o melhor *hardware* do mundo usando um sistema operacional defasado, tanto como um aplicativo de última geração tentar rodar em um *hardware* antigo e desatualizado.

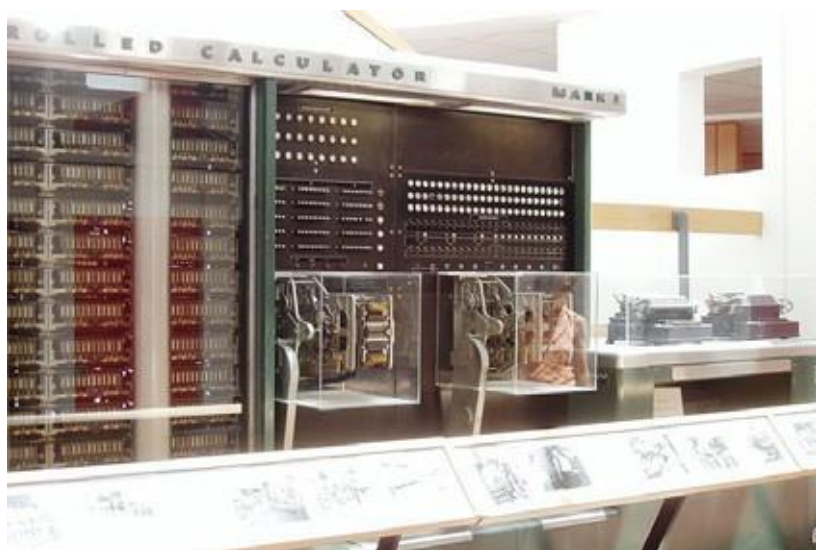
A evolução do *software* passa, também, pelas características históricas do *hardware*. Roger Pressman (2009), guru da Engenharia de *Software*, define o *software* como componentes não executáveis em máquina e componentes executáveis em máquina. Explica-nos que o *software* é criado por meio de uma série de regras que mapeiam as exigências de negócios que são desenvolvidos em linguagem de programação, que especifica a estrutura de seus dados, os atributos procedimentais e os requisitos relacionados para código e os converte em instruções executáveis em máquina.

O *software* engloba códigos que executam funções entre si e, normalmente, retornam alguma informação ao usuário. Para a criação do *software*, os programadores utilizam uma linguagem de programação, que é interpretada por um compilador, que a transforma para código binário, o qual é lido pelo *hardware*.

Para entendermos essa definição, segue um pouco da história da “era do computador”.

O primeiro computador surgiu para fins militares na década de 1940, o “Mark I” (tradução por tanque de guerra), foi financiado pela Marinha norte-americana em conjunto com a Universidade de Harvard e a IBM, onde ocupava, aproximadamente, 120 m³ e tinha 4,5 toneladas. Um gigante eletromagnético lançado no ano de 1944 para auxiliar nos cálculos de precisão necessários para balística (BERNARDES, 2015).

FIGURA 1 - MARK I

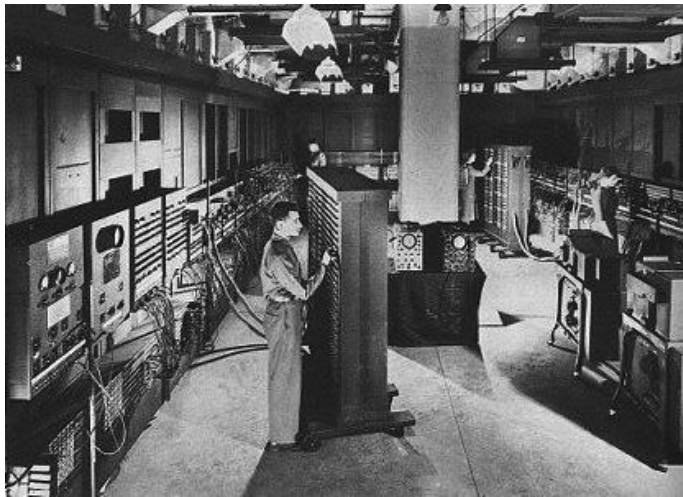


FONTE: Disponível em: <<http://www.geocities.ws/trabalhoweb2/>>. Acesso em: 03 jul. 2015

Nessa época, devido às limitações da “linguagem máquina”, as operações para realizar os processos eram complexas, demandando várias pessoas para manter o funcionamento dos computadores. No período, uma das criações mais notáveis – influenciando, inclusive, na Segunda Guerra Mundial – foi a máquina de Alan Turing, que foi capaz de decifrar os códigos utilizados pela Alemanha. Turing é considerado uma das maiores mentes na Ciência da Computação, tendo vários conceitos sendo estudados ainda nos dias atuais, como a Inteligência Artificial (IA).

Na mesma década, em 1946, surgiu o primeiro computador eletrônico à válvula (relés eletromagnéticos e máquinas perfuradoras de cartões), desenvolvido por Eckert e Mauchly, o “ENIAC” (*Electronic Numerical Integrator and Computer* ou Computador Integrador Numérico Eletrônico), cuja velocidade de processamento era superior à do Mark I e configurações mais próximas dos computadores atuais. Ocupava, aproximadamente, 180 m² e tinha 30 toneladas. Sua memória era muito pequena, e a cada nova operação era necessário reconfigurar toda a sua fiação, exigindo um enorme esforço humano.

FIGURA 2 - ENIAC

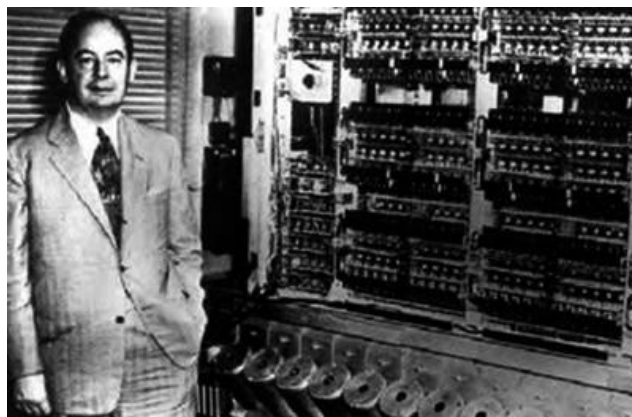


FONTE: Disponível em: <<http://www.geocities.ws/trabalhoweb2/>>. Acesso em: 03 jul. 2015

A principal mudança da primeira geração para a segunda foi a substituição da válvula para o transistor, deixando-se de utilizar a linguagem de máquina para a programação em Assembly, facilitada por dispositivos de entrada e saída, melhorando a comunicação homem-máquina.

Portanto, o primeiro computador eletrônico com programa armazenado foi o “EDVAC” (*Electronic Discrete Variable Automatic Computer* ou Computador Eletrônico com Discreta Variação Automática), desenvolvido por Von Neumann. Utilizava o sistema binário, uma máquina digital de armazenamento (“memória”) para comportar, respectivamente, instruções e dados na mesma unidade de processamento (CPU), podendo, assim, manipular tais programas. Tornou-se a arquitetura padrão para os computadores mais modernos, pois era dotado de cem vezes mais memória interna do que o ENIAC, um grande salto para a época, e ocupava 45,5 m² com quase oito toneladas (BERNARDES, 2015).

FIGURA 3 - EDVAC



FONTE: Disponível em: <<http://www.geocities.ws/trabalhoweb2/>>. Acesso em: 03 jul. 2015

Em 1949, Maurice Wilkes criou o EDSAC (*Electronic Delay Storage Automatic Calculator* ou Calculadora Automática com Armazenamento por Retardo Eletrônico), o primeiro computador operacional em grande escala capaz de armazenar seus próprios programas.

Na década de 50 surgiu o primeiro computador comercial do mundo, o LEO (Escritório Eletrônico de Lyons), que começou a automatizar os trabalhos de escritórios, criado através da parceria entre a J. Lyons e a Universidade de Cambridge e começou a ser utilizado em 1954.

FIGURA 4 - LEO



FONTE: Disponível em: <<http://www.geocities.ws/trabalhoweb2/>>. Acesso em: 03 jul. 2015

A partir de 1953, o uso de circuitos integrados diminuiu o tamanho dos computadores e a empresa IBM começou a comercializar o computador IBM 701, onde eram utilizados cartões perfurados para o armazenamento de programas e dados.

FIGURA 5 - CONSOLE DO IBM 701



FONTE: Disponível em: <<http://www.geocities.ws/trabalhoweb2/>>. Acesso em: 03 jul. 2015

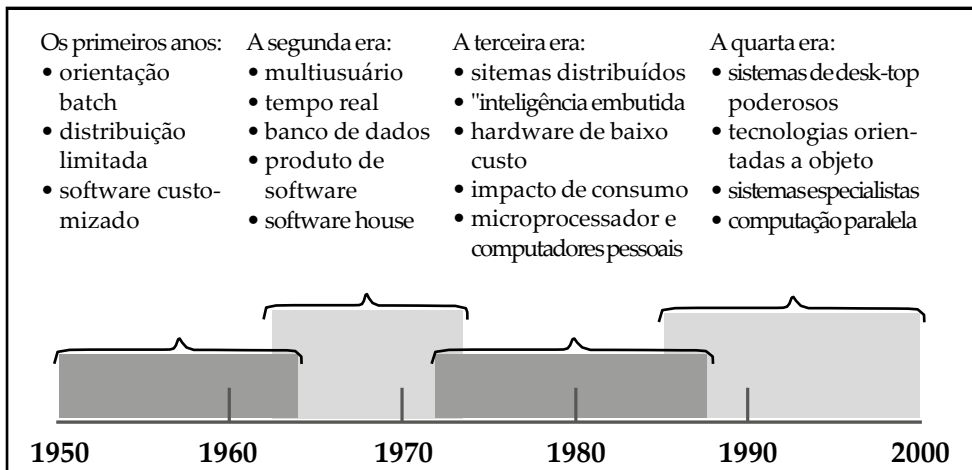
No final dos anos 60, o acesso a informações ficou mais rápido, foram introduzidas linguagens de alto nível, como Cobol e Fortran, complementadas por sistemas de armazenamento em disco e fitas magnéticas. Com o surgimento das linguagens imperativas Cobol e Fortran, as possibilidades de programação se expandiram vastamente.

O Cobol ainda é utilizado amplamente nos dias atuais em ambientes de *mainframe*, principalmente em bancos, devido à sua estabilidade, recebendo até expansão para ser utilizado como linguagem orientada a objeto. Outro ponto é o custo para realizar a migração e integração de dados para uma plataforma mais atual, que é muito alto.

2 EVOLUÇÃO DO SOFTWARE

Conforme visto, o desenvolvimento do *software* está estreitamente ligado há mais de cinco décadas de evolução do *hardware*, culminando em menor tamanho do *hardware*, fazendo com que sistemas baseados em computadores se tornassem mais sofisticados. Evoluímos dos processadores à válvula para os dispositivos microeletrônicos, que são capazes de processar milhares de instruções por segundo. A figura a seguir descreve a evolução do *software* dentro do contexto das áreas de aplicação de sistemas baseados em computador.

FIGURA 6 – A EVOLUÇÃO DO SOFTWARE



FONTE: Disponível em: <<http://www.batebyte.pr.gov.br/modules/conteudo/conteudo.php?conteudo=299>>. Acesso em: 03 de jul. 2015.

Conforme destacado por Azevedo (2015), a evolução do *software* possui as seguintes características:

- **Os primeiros anos:** como já visto anteriormente, no início, o *hardware* sofreu contínuas mudanças e o *software* foi incorporando-se aos poucos nesta evolução, na qual o *hardware* dedicava-se à execução de um único programa que, por sua vez, dedicava-se a uma única aplicação específica. Tinha-se uma

distribuição limitada e com poucos métodos sistemáticos. Como característica usava uma orientação *batch* (em lote) para a maioria dos sistemas. A maior parte dos *softwares* era desenvolvida e usada pela própria empresa e quem escrevia e colocava em funcionamento também tratava dos defeitos, e por conta do ambiente de *software* personalizado, o projeto era um processo implícito, realizado no cérebro de alguém, e a documentação muitas vezes não existia.

- **A segunda era:** entre a década de 1960 até o final da década de 1970 foi o período da multiprogramação. Os sistemas multiusuários ofereceram sofisticação de *software* e *hardware*, melhorando a interação homem-máquina. Houve o surgimento de sistemas de tempo real que podiam coletar, analisar e transformar dados de múltiplas fontes. Os avanços da armazenagem *online* levaram à primeira geração de sistemas de gerenciamento de banco de dados e surgimento dos *softwares houses*. Os programas para *mainframes* e minicomputadores eram distribuídos para centenas e, às vezes, milhares de usuários, e à medida que cresciam, devido à natureza personalizada de muitos programas, tornava-os virtualmente impossíveis de sofrerem manutenção. Uma "crise de *software*" agigantou-se no horizonte.

NOTA

- **A terceira era:** começou em meados da década de 1970 e continua até hoje. Existência dos sistemas distribuídos e múltiplos computadores, onde cada um, executando funções concorrentemente e comunicando-se um com o outro, aumenta intensamente a complexidade dos sistemas baseados em computador. As redes globais, as comunicações digitais de largura de banda elevada e a crescente demanda de acesso "instantâneo" a dados exigem muito dos desenvolvedores de *software*. Foi caracterizada, também, pelo advento e o generalizado uso de microprocessadores, computadores pessoais e poderosas estações de trabalho "*workstations*" de mesa. O microprocessador gerou um amplo conjunto de produtos inteligentes. Do automóvel a fornos micro-ondas, de robôs industriais a equipamentos para diagnóstico de soro sanguíneo. Em muitos casos, a tecnologia de *software* está sendo integrada a produtos, por equipes técnicas que entendem de *hardware*, mas que frequentemente são principiantes em desenvolvimento de *software*. O *hardware* de computador pessoal está se tornando rapidamente um produto primário, enquanto o *software* oferece a característica capaz de diferenciar.

- **A quarta era:** esta era está apenas começando. As tecnologias orientadas a objetos, orientadas a documentos, estão ocupando o lugar das abordagens mais convencionais para o desenvolvimento de *software* em muitas áreas de aplicação. As técnicas de "quarta geração" para o desenvolvimento de *software* já estão mudando a maneira segundo a qual alguns segmentos da comunidade de *software* constroem programas de computador. Os sistemas especialistas e o *software* de inteligência artificial finalmente saíram do laboratório para a aplicação prática em problemas de amplo espectro do mundo real. O *software* de rede neural artificial abriu excitantes possibilidades para o reconhecimento de padrões e para capacidades de processamento de informações semelhantes às humanas.
- **Era atual:** a sofisticação do *software* ultrapassou nossa capacidade de construir um *software* que extraia o potencial do *hardware*. A capacidade de construir programas não acompanha o ritmo da demanda de novos programas, a capacidade de manter os programas existentes é ameaçada por projetos ruins e recursos inadequados e as práticas de engenharia de *software* se fortalecem.

3 TIPOS DE SOFTWARE DO PONTO DE VISTA DA ENGENHARIA

O *software* é um conjunto de algoritmos codificados que permite ao computador executar uma operação ou um conjunto de operações culminando em tarefas. Analisaremos, aqui, os tipos de *software* disponíveis, bem como a função e utilidade desses tipos de *software*.

Segundo Azevedo (apud VERZELLO, 1984), o *software* é classificado em três tipos: (1) *Software* de sistema, que são programas escritos para controlar e coordenar o *software*; (2) *Software* de linguagens, que são programas que traduzem outros programas para a forma binária, que é a linguagem utilizada pelos componentes do sistema computacional para mantê-los salvos em bancos de dados especiais; e, por último, (3) *Software* de aplicação, que são programas escritos para resolver problemas comerciais ou prestar outros serviços de processamento de dados aos usuários.

Roger Pressman (1995), porém, amplia essa classificação de *software* em sete diferentes categorias, afirmando ser uma tarefa um tanto difícil desenvolver categorias genéricas para aplicações de *softwares*, pois, no mesmo passo que o *software* cresce, desaparece a visão de compartimentos:

1. **Software Básico:** é uma coleção de programas que dão apoio a outros programas. É caracterizado pela forte interação com *hardware*, intenso uso por múltiplos usuários; operações concorrentes que exigem escalonamento *schedule*; compartilhamento de recursos e sofisticada administração do processo; estruturas de dados complexas e múltiplas interfaces externas. Exemplo: compiladores, editores simples, *drivers*, componentes do SO.
2. **Software de Tempo Real:** monitorar, analisar e controlar eventos do mundo real, caracterizado pela coleta de dados do ambiente externo, análise que transforma a informação de acordo com a necessidade do sistema, controle e saída para um ambiente externo e um componente de monitoração que

coordena todos os outros. O termo tempo real difere de interativo ou tempo compartilhado, pois deve responder dentro de restrições de tempos exatos sem resultados desastrosos.

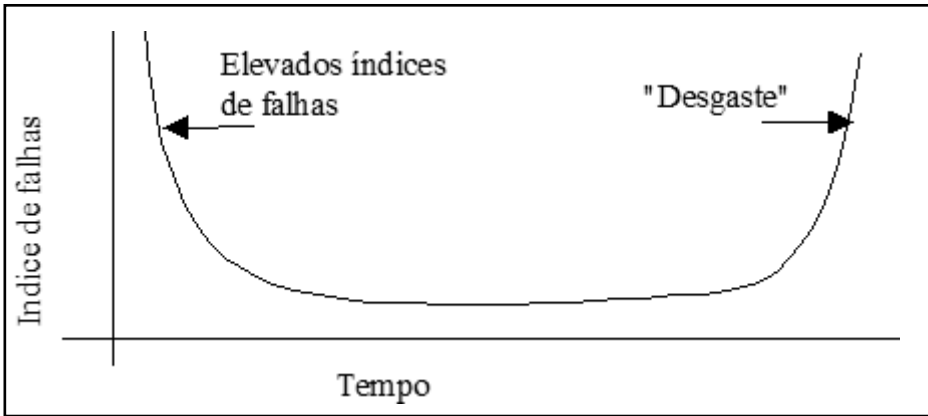
3. **Software Comercial:** facilita as operações comerciais e decisões administrativas. As aplicações dessa área reestruturam os dados de uma forma que facilita as operações comerciais e as tomadas de decisões administrativas. Além da aplicação de processamento de dados convencional, as aplicações de *software* comerciais abrangem a computação interativa. Exemplos: controle de estoque, finanças, vendas etc.
4. **Software Científico e de Engenharia:** algoritmos com intenso processamento de números e cálculos. As aplicações diversificadas variam da análise de fadiga mecânica de automóveis à dinâmica orbital de naves espaciais recuperáveis, e da biologia molecular à manufatura automatizada. Exemplos: sistemas de astronomia, naves espaciais, matemática avançada etc.
5. **Software Embutido:** é usado para controlar produtos e sistemas para mercados industriais e de consumo, pode utilizar memória de somente leitura e usa rotinas limitadas e particulares. O *software* embutido reside na memória só de leitura (*read only*) e pode executar funções limitadas e particulares (por exemplo, controle de teclado para fornos de micro-ondas) ou oferecer recursos funcionais de controle significativos (por exemplo, funções digitais em automóveis, tais como controle, mostradores no painel, sistemas de freio etc.).
6. **Software de computador pessoal:** utilizados em computadores de uso pessoal. Exemplos: editores de texto, planilhas, calculadora, jogos, computação gráfica, gerenciamento de dados, aplicações financeiras pessoais e comerciais, redes externas ou acesso a banco de dados, são apenas algumas das centenas de aplicações.
7. **Software de inteligência artificial:** faz uso de algoritmos não numéricos para resolver problemas complexos que não sejam favoráveis à computação ou à análise direta. Atualmente, a área de *Artificial Inteligency* (AI) mais ativa é a dos sistemas especialistas baseados em conhecimentos, porém outras áreas de aplicação para o *software* de AI são o reconhecimento de padrões (voz e imagem), jogos e demonstração de teoremas. Uma rede neural simula a estrutura dos processos cerebrais (a função do neurônio biológico) e pode levar a uma nova classe de *software* que consegue reconhecer padrões complexos e aprender com a experiência passada. Exemplos: sistema de reconhecimento de imagem, sistemas especialistas, redes neurais e aprendizado etc.

Diante do contexto histórico da evolução do computador, podemos perceber que o *software* é um elemento de sistema lógico e complexo, projetado pela engenharia, e o *hardware*, um elemento físico projetado por manufatura. Ambos apresentam características diferentes quanto à forma de utilização. Por exemplo: com o passar dos tempos, o *hardware* vai desgastando-se, exigindo manutenção, e o *software*, por sua vez, deteriora-se, exigindo sua evolução (vide Figura 7).

Os índices de falhas são relativamente elevados logo no começo do ciclo de vida do *hardware*, os defeitos são corrigidos e o índice de falhas cai para estável,

porém, à medida que o tempo passa, o índice de falhas eleva-se novamente, conforme os componentes de *hardware* vão sofrendo males ambientais (poeira, vibração, abuso, temperaturas extremas etc.).

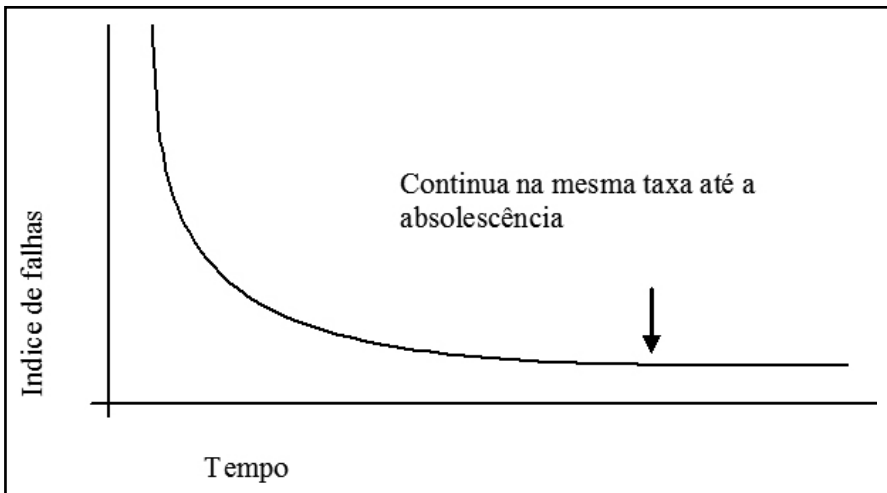
FIGURA 7 - CURVAS DE FALHAS PARA O *HARDWARE*



FONTE: Disponível em: <<http://blog.tecsystem.com.br/index.php/erros-e-acertos-no-desenvolvimento-de-software/>>. Acesso em: 05 jul. 2015.

Já o *software* não é sensível aos problemas ambientais. Os defeitos não descobertos no começo da vida de um programa provocarão elevados índices de falhas; após corrigidas, a curva achata-se; entretanto, fica claro que o *software* não se desgasta. Todavia, se deteriora!

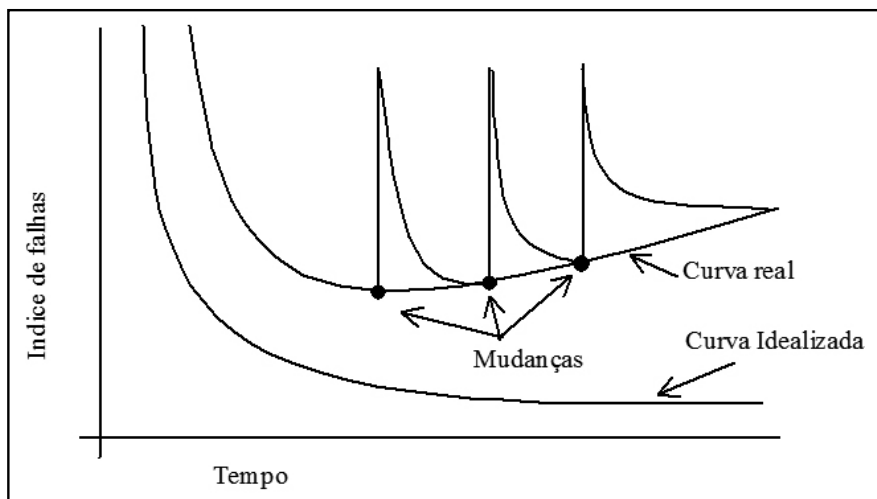
FIGURA 8 - CURVA DE FALHAS DO *SOFTWARE* (IDEALIZADA)



FONTE: Disponível em: <<http://blog.tecsystem.com.br/index.php/erros-e-acertos-no-desenvolvimento-de-software/>>. Acesso em: 05 jul. 2015.

Portanto, segundo Amparo (2015), esta aparente contradição pode ser mais bem explicada considerando a Figura 9. Durante sua vida, o *software* enfrentará mudanças (manutenção). Quando estas são feitas, é provável que novos defeitos sejam introduzidos, fazendo com que a curva do índice de falhas apresente picos.

FIGURA 9 - CURVA DE FALHAS REAL PARA O SOFTWARE



FONTE: Disponível em: <<http://blog.tecsystem.com.br/index.php/erros-e-acertos-no-de-senvolvimento-de-software/>>. Acesso em: 05 jul. 2015

Percebe-se que a manutenção do *software* envolve consideravelmente mais complexidade do que a de *hardware*. Basta substituir um componente de *hardware* por uma peça de reposição; já com o *software* não existe reposição de peças, toda falha indica um erro de projeto ou no processo por meio do qual o projeto é especificado e traduzido em código executável por máquina.

É necessário antecipar os caminhos em que o *software* sofre mudanças, dessa forma pode-se modificá-lo mais facilmente para acomodar tais necessidades. Porém, antecipar-se às mudanças não é uma tarefa fácil, uma vez que existem muitas razões pelas quais os sistemas mudam.

IMPORTANTE

Furtado (2007) conclui que, com o passar do tempo, os *softwares* vão “envelhecendo”, sua estrutura se torna ultrapassada e, por já não satisfazerem suas tarefas como antes, são descartados. O *software* é inerentemente flexível e pode ser alterado. Raramente é necessário descartar um *software* inteiro por ele

não ser capaz de realizar determinada tarefa ou atingir um novo resultado. Em determinadas circunstâncias é preferível adaptar o *software* ao invés de eliminá-lo totalmente. Porém, para que isso seja possível é necessário que sejam adotadas técnicas de evolução de *software*, dessa forma o processo de reestruturação de um *software* pode ser possível, evitando que ele perca sua utilidade.

Estas técnicas de evolução devem auxiliar o *software* a não sofrer tantos impactos após sucessivas versões. Como as modificações estão presentes na vida de um *software*, faz-se necessário utilizar controles a fim de reverter o envelhecimento do *software*, ou seja, adiar o dia em que este já não poderá ser mais viável.

RESUMO DO TÓPICO 1

Neste tópico, você aprendeu que:

- O *hardware* e o *software* sempre andaram em paralelo, para um funcionar precisava da evolução dos recursos do outro. O *software* é definido como componentes não executáveis em máquina e componentes executáveis em máquina.
- A história do computador se iniciou em 1940, com o Mark I, feito para fins militares; em seguida, em 1946, surgiu o primeiro computador à válvula, o ENIAC (Computador Integrador Numérico Eletrônico), que tinha mais velocidade de execução. Na mesma década surgiram, também, o primeiro computador eletrônico com programa armazenado, o EDVAC (Computador Eletrônico com Discreta Variação Automática) e o primeiro computador operacional, o EDSAC (Calculadora Automática com Armazenamento por Retardo Eletrônico). Por fim, na década de 50, o computador comercial, o LEO (Escritório Eletrônico de Lyons), denominado IBM 701.
- Conforme já visto, o *software* veio para controlar o *hardware* e ajudar o usuário em suas tarefas. Inicialmente, preso a tarefas mais industriais até chegar no uso do Computador Pessoal.
- De modo geral, as áreas de aplicações da evolução do *software* são:
 - o Os primeiros anos: a partir de 1950 eram orientados a *batch*, distribuição limitada e *software* customizado.
 - o A segunda era: a partir de 1960, o *software* multiusuário, tempo real, banco de dados, produção de *software* e *software house*.
 - o A terceira era: a partir de 1970, os sistemas distribuídos, *hardware* de baixo custo, microprocessadores e computadores pessoais.
 - o A quarta geração: a partir de 1980, os sistemas *desktop* poderosos, tecnologia orientada a objeto, sistemas especialistas e computação paralela.
- Robert J. Verzello classifica o *software* pelos tipos: *software* de sistemas, *software* de linguagens e *software* de aplicações. Já Roger Pressman (2002) o classifica pelos tipos: *software* básico, *software* de tempo real, *software* comercial, *software* científico e de engenharia, *software* embutido, *software* de computador pessoal e *software* de inteligência artificial.

- Finalizando, o *software* é um elemento de sistema lógico e complexo projetado pela engenharia e o *hardware* um elemento físico projetado por manufatura. Com o passar dos tempos, o *hardware* foi desgastando-se devido ao ambiente físico, exigindo a manutenção, e o *software*, por sua vez, se deteriora ou fica defasado, exigindo sua evolução.

TÓPICO 2

FUNDAMENTOS DE ENGENHARIA DE SOFTWARE

1 INTRODUÇÃO

A Engenharia de *Software* originou-se conceitualmente em 1969, por Fritz Bauer, durante uma conferência patrocinada pelo Comitê de Ciência da Organização do Tratado do Atlântico Norte (OTAN), período da segunda era da evolução do *software* e momento em que a crise do *software* precisava de uma solução para que em seu desenvolvimento os projetos de *software* fossem entregues dentro de custo e prazo adequados.

Segundo Hirama (2011), nessa época a crise foi identificada pela preocupação crescente na comunidade de *software* com a quantidade de defeitos, entregas fora de prazo e altos custos do *software*. Os códigos eram difíceis de manter pela inexistência de métodos eficazes para seu desenvolvimento. Não existia solução eficaz que evitasse tantos “furos” nos projetos. A falta de qualidade do *software* não era evitada pelos desenvolvedores, porque ainda não se usava controle de qualidade no ambiente de desenvolvimento, sendo difíceis, também, de se manter e evoluir.

O termo “crise do *software*” foi usado pela primeira vez com impacto por Dijkstra (1972), o qual avaliava que, considerando o rápido progresso do *hardware* e das demandas por sistemas cada vez mais complexos, os desenvolvedores simplesmente estavam se perdendo, porque a Engenharia de *Software*, na época, era uma disciplina incipiente.

Como mencionado nesta mesma obra de Hirama (2007, p. 7), Fritz Bauer teria dito: “A Engenharia de *Software* é o estabelecimento e uso de sólidos princípios de engenharia a fim de obter um *software* que seja confiável e que funcione de forma econômica e eficiente em máquinas reais”.

Em seu livro *Engenharia de Software: Qualidade e Produtividade com Tecnologia*, Hirama (2011) explica que a existência da Engenharia de *Software* se faz importante pelos seguintes motivos:

- **A complexidade dos softwares:** especificar sistemas é uma atividade bastante complexa. Não se trata apenas de fazer uns “**programinhas**”.
- **Insatisfação dos usuários:** o usuário precisa de sistemas funcionando de acordo com suas necessidades e que sejam fáceis de serem operados e/ou cujo desenvolvimento não seja demorado.

- **Produtividade:** costuma estar quase sempre abaixo do desejado. Frequentemente, a alocação de recursos e atividades é desbalanceada. Custos, tempo e recursos geralmente são subestimados.
- **Confiabilidade do Sistema:** há diversas estatísticas que provam a pouca confiabilidade de boa parte dos sistemas. Não basta que o sistema produza resultados solicitados pelo usuário, mas que também tenha o desempenho adequado.
- **Manutenibilidade:** facilidade de se modificar um sistema para adaptar-se a circunstâncias novas, inexistentes à época da implantação. Sistemas recentemente implantados são substituídos por novos devido ao alto custo para sua manutenção.

Com minha experiência desde 2007 como gestor do processo de Engenharia de *Software*, posso reforçar que a Engenharia de *Software* é uma aliada indispensável às empresas de *software*, é o que define métodos sistemáticos para o desenvolvimento de *software*, buscando melhorar e amadurecer as técnicas e ferramentas utilizadas no ambiente de desenvolvimento para aumentar sua produtividade e qualidade de desenvolvimento.

Não é fácil desenvolver um *software* de qualidade, por isso é preciso criar uma disciplina aplicada a toda a equipe envolvida, começando pelos gerentes de tecnologia, diretores, analistas, programadores e a equipe de suporte e usuários do sistema. Empresas que desenvolvem *software* de qualidade são mais competitivas e podem, em geral, oferecer um melhor serviço ao seu cliente final.

IMPORTANTE

Segundo Ian Sommerville (2011), a Engenharia de *Software* é uma disciplina da engenharia de sistemas que se ocupa de todos os aspectos da produção de *software*, desde os estágios iniciais de levantamento e especificação de requisitos até a implantação e manutenção, ou seja, que entrou em operação. É um conjunto de atividades, parcialmente ou totalmente ordenadas, com a finalidade de obter um produto de *software* de qualidade e cumprir corretamente os contratos de desenvolvimento.

Segundo Roger Pressman (2006), a Engenharia de *Software* poderá ser mais bem entendida como uma tecnologia em camadas ou níveis, conforme pode ser vista na Figura 10.

FIGURA 10 - CAMADAS DA ENGENHARIA DE SOFTWARE



FONTE: Disponível em: <http://3.bp.blogspot.com/_CMoqSGzMYOg/SkrJEy3nGel/AAAAA-AAAAAAk/Ee1ZgJBwMdm/s320/Figura+2.1.Engenharia+de+Software+em+Camadas.gif>. Acesso em: 06 jul. 2015

Na base da figura, formando a camada foco na qualidade, dá-se ênfase à preocupação de qualquer disciplina de engenharia, que é qualidade. A qualidade na Engenharia de *Software* é baseada nos conceitos de gerenciamento de qualidade total (TQM – *Total Quality Management*) para a melhoria contínua dos processos. O TQM é uma abordagem de gerenciamento organizacional (princípios, métodos e técnicas) para obter sucesso em longo prazo através da satisfação dos clientes.

A camada de processo permite integrar as camadas de métodos e de ferramentas para que se possa desenvolver um *software* nos prazos acordados e de maneira adequada. Um processo permite que se planeje e se controle projeto de *software*.

A camada de métodos provê as abordagens e as atividades necessárias para a construção de um *software*. Os métodos abrangem um conjunto amplo de tarefas que incluem análise de requisitos, projeto, implementação, testes e manutenção. Os métodos de Engenharia de *Software* são baseados em um conjunto de princípios que governam cada área de tecnologia e incluem atividades de modelagem e técnicas descritivas.

A camada de ferramentas provê apoio automatizado ou semiautomatizado para processos e métodos. As ferramentas da área de Engenharia de *Software* são conhecidas como CASE (Engenharia de *Software* Apoiada por Computador, do termo em inglês *Computer-Aided Software Engeneering*).

FONTE: Hirama (2011, p. 8)

2 PRINCÍPIOS DA ENGENHARIA DE SOFTWARE

A aplicação da Engenharia de *Software* segue uma abordagem bem completa, onde são recomendados caminhos como referência para sua correta e benéfica utilização. Em diversos livros são apresentados como introdução à Engenharia de *Software* alguns princípios gerais aplicados durante toda a fase de desenvolvimento de *software*, da importância de algumas propriedades gerais dos processos e produtos.

A seguir são mencionados, de forma resumida, os conceitos de cada um dos 12 princípios da Engenharia de *Software* descritos por Carvalho (2001) em seu livro *Introdução à engenharia de software*:

1. **Formalidade:** deve ser desenvolvido com passos definidos e com precisão, seguidos de maneira efetiva. Não se deve restringir a criatividade, mas melhorá-la, uma vez que são criticamente analisados à luz de uma avaliação formal. Seus efeitos benéficos podem ser sentidos na manutenção, reutilização, portabilidade e entendimento do *software*.
2. **Abstração:** é o processo de identificação dos aspectos importantes de um determinado fenômeno, ignorando-se os detalhes. Os programas, por si só, são abstrações das funcionalidades do sistema.
3. **Decomposição:** uma das maneiras de lidar com a complexidade é subdividir o processo em atividades específicas, diminuindo a complexidade do problema, provavelmente atribuídas a especialistas de diferentes áreas. A decomposição das atividades leva, também, à separação das preocupações ou responsabilidades.
4. **Generalização:** pensar na resolução de uma forma generalizada para permitir reutilização. É o processo de identificação dos aspectos importantes de um determinado fenômeno, ignorando-se os detalhes.
5. **Flexibilidade:** diz respeito tanto ao processo como ao produto do *software*. O produto sofre constantes mudanças, pois, em muitos casos, a aplicação é desenvolvida incrementalmente enquanto seus requisitos ainda não foram totalmente entendidos. Devem permitir ao processo de desenvolvimento que o produto possa ser modificado com facilidade. Permitir que partes ou componentes de um produto desenvolvido possam ser utilizados em outros sistemas, bem como a sua portabilidade para diferentes sistemas computacionais.
6. **Padronização:** padronizar o processo de construção do *software* para facilitar o entendimento e manutenção.
7. **Rastreabilidade:** modo de saber o que já foi feito e o que ainda não foi feito.
8. **Desenvolvimento iterativo:** toda a equipe é engajada na solução. Exemplo: SCRUM.
9. **Gerenciamento de requisitos:** deixar claro o que deve ser atendido pelo sistema formalmente em requisitos.
10. **Arquiteturas baseadas em componentes:** separar a solução em componentes bem definidos funcionais e lógicos, com interfaces bem definidas que não compartilham estados e se comunicam por troca de mensagens contendo dados.

11. **Modelagem visual:** diagramas de fácil visualização da solução de um determinado problema. Utilizado para rápida compreensão da solução ou de um processo.
12. **Verificação contínua de qualidade:** criação de testes automatizados de modo que garantam a cobertura do código e testes de qualidade que garantam a qualidade do *software*.

IMPORTANTE

Portanto, como a tecnologia não para de crescer, é preciso buscar implementar metodologias ao ambiente de desenvolvimento de *software* nas empresas que definam atividades e técnicas a serem utilizadas em sua criação, oferecendo modelos, padrões, arquiteturas, métodos e processos que possam oferecer maior produtividade, eficiência, profissionalismo e criatividade das equipes envolvidas. Enfim, não deve se preocupar em apenas criar fórmulas e regras para chegar ao objetivo final do projeto, mas em realizar isso da melhor forma e com resultados positivos, conforme planejamento e execução destas metodologias.

Com o avanço das tecnologias e o aumento da complexidade dos *softwares*, as exigências foram aumentando e, conseqüentemente, as ferramentas de desenvolvimento foram surgindo e melhorando em relação à estruturação na criação dos sistemas através de suas metodologias de desenvolvimento, que serão abordadas no tópico seguinte.

3 METODOLOGIA DE DESENVOLVIMENTO DE SISTEMAS

No início da era da computação não existiam métodos para o controle e a produção do *software*, pois eram mais simples de serem feitos e não eram produzidos em grande escala. Também não existiam equipes para realizar um planejamento do desenvolvimento de *software*, o que influenciava diretamente em constantes desvios no prazo de entrega e no alto custo de produção do *software*.

Segundo Fernandes (1999), metodologia de sistemas se define como um conjunto de normas, procedimentos, técnicas e ferramentas de análise que definem o padrão desejado por uma empresa para o desenvolvimento de projetos de sistemas.

A ausência de uma metodologia de desenvolvimento de sistemas pode levar ao caos, na medida em que cada indivíduo procura aplicar em seu projeto as melhores soluções dentro das limitações de sua experiência profissional. Mesmo que suas soluções sejam ótimas e que os resultados individuais sejam melhores, dificilmente, no conjunto de todas as aplicações de uma corporação, haverá a harmonia desejada. A produtividade e a eficiência que são esperadas de um departamento de sistemas não podem ser obtidas sem critérios, sem regras e sem análise contínua das ferramentas de trabalho postas à disposição dos profissionais de sistemas.

FONTE: Fernandes (1999)

A Figura 11 apresenta que estes métodos estabelecem produtos de trabalho padronizados que facilitam as atividades de manutenção de *software*, permitindo que os colaboradores sejam treinados, melhorando a qualidade dos serviços através de um canal de comunicação uniforme entre os membros da equipe de desenvolvimento, para um maior aproveitamento dos seus recursos.

FIGURA 11 - O QUE É UMA METODOLOGIA



FONTE: Disponível em: < <http://slideplayer.com.br/slide/386223/>>. Acesso em: 24 jul. 2015

Em um projeto de *software* existem, em geral, muitos profissionais envolvidos. Têm-se, entre outros, gerentes, analistas, arquitetos, programadores e testadores. Durante a realização das atividades de desenvolvimento, a comunicação entre eles é fundamental. Para estabelecer um canal de comunicação uniforme, é necessário aplicar métodos definidos em processos de desenvolvimento de *software*.

FIGURA 12 - PROCESSO DE COMUNICAÇÃO



FONTE: O autor

IMPORTANTE

As ferramentas CASE (*Computer Aided Software Engineering* ou Engenharia de Software apoiado por Computador) são programas que auxiliam atividades de engenharia de software na construção de sistemas. Desde a análise de requisitos e modelagem até programação e testes. Para identificar a “Seleção de Ferramentas CASE” recomenda-se a leitura da monografia de Elison Roberto Imenes, disponível em: <<https://docplayer.com.br/16002959-Selecao-de-ferramentas-case.html>>.

Fernandes (1999) nos diz que, para que uma metodologia de desenvolvimento de sistemas seja consistente, oferecendo maior produtividade e qualidade, deverá atender a alguns requisitos fundamentais:

- **Padronização:** executar as atividades de maneira idêntica, fazendo com que haja aperfeiçoamento do processo.
- **Flexibilidade:** é a capacidade de se adaptar às mudanças.
- **Documentação:** manter informações sobre o produto e garantir rapidez diante das mudanças.
- **Modularização:** consiste em dividir um conjunto de atividades em vários conjuntos menores, objetivando melhor visualização e acompanhamento por parte de todos os interessados no resultado final.
- **Planejamento:** forma madura de administrar o tempo, é programar o futuro em relação às metas e aos objetivos a serem alcançados.

Gomede (2010) menciona a existência da divisão da Engenharia de *Software* em dez áreas de conhecimento, segundo o SWEBOK, as quais são descritas a seguir:

- **Design de Software:** Área que envolve definição da arquitetura, componentes, interfaces e outras características de um componente ou sistema. Analisando como um processo, esta é uma etapa do ciclo de vida da ES, onde é processada a análise dos requisitos com o objetivo de produzir uma descrição da arquitetura do *software*, ou seja, de forma interativa, os requisitos são traduzidos em um documento para construção do *software*.
- **Construção de Software:** Trata-se da implementação ou codificação do *software*, verificação, testes de unidade, testes de integração e depuração. Está envolvida com todas as áreas de conhecimento, porém, ligada fortemente às áreas de *Design* e *Teste de Software*, pois este processo de construção abrange significativamente ambas as áreas.
- **Teste de Software:** É uma atividade com o intuito de avaliar a qualidade do produto, buscando identificar problemas e defeitos existentes. Trata-se de um elemento crítico da garantia da qualidade de *software* e representa a verificação final da especificação, projeto e geração de código.
- **Manutenção de Software:** Área definida como o conjunto das atividades requeridas para fornecer apoio a um sistema de *software*, que pode ocorrer antes ou depois da entrega. São realizadas atividades de planejamento antes da entrega do *software*; depois, são executadas modificações com o objetivo de melhorar o desempenho, corrigir falhas, ou adaptá-las a um ambiente externo. Podemos chamar de “ajuste fino”.
- **Gerenciamento de Configuração de Software:** É um processo que provê recursos para o controle da evolução, identificação e auditoria dos artefatos de *software* gerados durante o desenvolvimento do projeto, ou seja, é o controle de versões do *software*, com a finalidade de estabelecer e manter a integridade dos produtos de *software* durante todo seu ciclo de vida.
- **Gerenciamento de Engenharia de Software:** Pode-se definir como a aplicação das atividades de gerenciamento, garantindo que o desenvolvimento e a gerência de *software* sejam sistemáticos, disciplinados e qualificados. Englobando atividades como controle, documentação, monitoramento e medição. Em suma, é a coordenação e o planejamento.
- **Engenharia de Processo de Software:** Seu objetivo é implementar processos novos e melhores, seja no escopo individual, de projeto ou organizacional, e pode ser definida como uma visão geral sobre questões do processo, amplamente relacionadas à definição, implementação, avaliação, mensuração, gerenciamento, mudanças e melhorias do processo de ciclo de vida de *software*.
- **Ferramentas e Métodos de Software:** São ferramentas criadas para prestar auxílio no ciclo de vida do *software*. Ferramentas estas que normalmente automatizam atividades do processo de desenvolvimento, auxiliando o analista no processo de concentração para as atividades que exigem

maior trabalho intelectual. Estruturam atividades de desenvolvimento e manutenção de *software* para torná-las sistemáticas e suscetíveis ao sucesso, já que seu objetivo é a pesquisa de ferramentas e métodos para o aumento da produtividade e redução de falhas.

- **Qualidade de Software:** Está diretamente ligada à qualidade de que o *software* é submetido durante o processo de desenvolvimento, consequentemente, para a qualidade existir, o processo de desenvolvimento de um produto de *software* precisa ser bem definido, documentado e acompanhado. A avaliação da qualidade normalmente é feita através de modelos. Modelos estes que descrevem e alinham as propriedades de qualidade do produto. Os modelos de avaliação mais aceitos e usados no mercado são CMMI (*Capability Maturity Model Integration*), proposto pelo CMM (*Capability Maturity Model*) e a norma ISO/IEC 9126, proposta pela ISO (*International Organization for Standardization*).
- **Requisitos de Software:** Requisitos de *software* expressam a necessidade e restrições ou limitações colocadas sobre o produto ou *software* que auxiliam na solução de problemas do mundo real, expondo, analisando, especificando e validando os requisitos de *software*. Ananias (2009) também menciona que a gerência de requisitos concorre em paralelo com a engenharia de requisitos e se faz necessária para minimizar os problemas que podem ocorrer em todos os estágios do desenvolvimento de sistemas, sendo uma das mais importantes áreas da engenharia de *software*.

FONTE: Disponível em: <<http://evertongomede.blogspot.com.br/2010/08/areas-de-conhecimento-segundo-o-swebok.html>>. Acesso em: 06 jul. 2015.

3.1 FASES DE DESENVOLVIMENTO DE SOFTWARE

Atualmente, muitas são as metodologias de desenvolvimento de *softwares*. Existem as clássicas (antigas), que são mais estáveis de serem executadas através de diversos ciclos de vida prescritivo, seguindo um único caminho de trabalho, e aquelas metodologias ágeis, que possuem diversas formas dinâmicas de execução, exigindo maior experiência dos envolvidos.

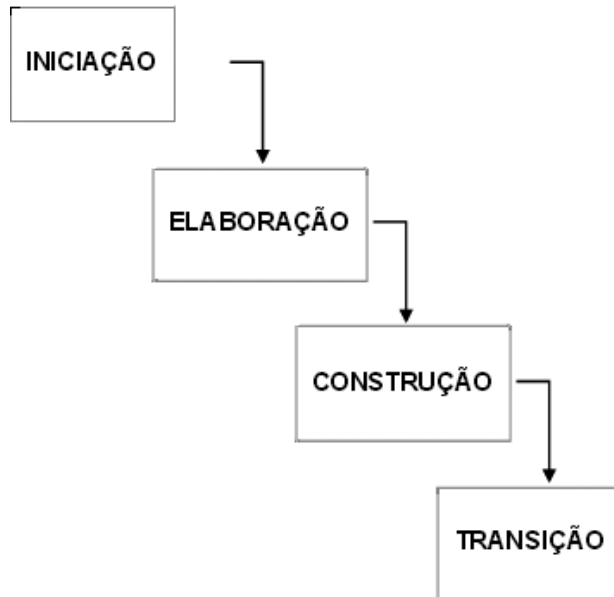
Para capturar a dimensão do tempo de um projeto, o processo de Engenharia de *Software* se divide em quatro fases que indicam a ênfase que é dada no projeto em um dado instante:

- **Fase de Iniciação:** ênfase no escopo.
- **Fase de Elaboração:** ênfase na análise.
- **Fase de Construção:** ênfase no desenvolvimento.
- **Fase de Transição:** ênfase na implantação.

Podemos colocar, ainda, que a fase de iniciação foca no tratamento de riscos relacionados com os requisitos de negócio; na fase de elaboração, o foco deve ser nos riscos técnicos e arquiteturais. O escopo deve ser revisado e os requisitos devem estar mais compreendidos do que durante a construção, a atenção será

voltada para os riscos lógicos e a maior parte do trabalho será realizada; e na fase de transição, finalmente, serão tratados os riscos associados com a logística de distribuição do produto para a base de usuários.

FIGURA 13 - FASE DE DESENVOLVIMENTO DE SOFTWARE



FONTE: O autor

Segue uma breve descrição das fases, colocando os principais itens a serem atendidos, bem como quais os principais produtos gerados em cada fase, onde em qualquer fase poderá ocorrer um replanejamento ou mudança de escopo, conforme monitoramento e controle realizados pelos coordenadores ou gerentes de projetos.

3.1.1 Fase de iniciação

- Esta fase tem como objetivo principal o planejamento do projeto e compreensão real do escopo do projeto.
- Nesta fase serão envolvidos, principalmente, os seguintes atores com suas respectivas responsabilidades:
 - o Coordenador de Projeto: com a responsabilidade de elaborar e aprovar o planejamento do Projeto (o principal documento gerado é o Plano de Projeto (com seus anexos)).
 - o Analista de sistema: terá a responsabilidade de refinar os Requisitos de Sistema para garantir o escopo do projeto.
 - o Analista de Negócio: tem a responsabilidade de aprovar os Requisitos de Sistema que irão garantir o entendimento do escopo do projeto.

- Ao final desta fase deve-se garantir que planejamento do projeto e o escopo do projeto foram compreendidos por todos os envolvidos e aprovados por todos os responsáveis.

3.1.2 Fase de elaboração

- Esta fase tem como objetivo principal a realização da análise do projeto, onde serão criados os documentos que definirão como o projeto será implementado.
- Nesta fase será envolvido, principalmente, o seguinte ator com suas respectivas responsabilidades:
 - o Analista de Sistema terá responsabilidade de definir como os requisitos de sistemas devem ser implementados, apresentando soluções para regras de negócio, banco de dados etc. O analista poderá representar a solução através do Documento de Caso de Uso e/ou documento de Especificação de Implementação, entre outros.
- Ao final desta fase deve-se garantir que a solução para análise está representada da melhor forma, que irá possibilitar ao Programador realizar seu trabalho, bem como ao Analista de Teste.

3.1.3 Fase de construção

- Esta fase tem como objetivo principal a materialização da análise, pois será realizada a implementação dos componentes que irão compor o projeto. Nesta fase, também, serão realizados os testes.
- Nesta fase será envolvido, principalmente, o seguinte ator com suas respectivas responsabilidades:
 - o Implementador: terá responsabilidade de implementar os componentes e realizar os testes de unidade. O principal produto gerado será o código-fonte.
- Ao final desta fase deve-se garantir que os requisitos solicitados foram implementados conforme a solicitação.

3.1.4 Fase de transição

- Esta fase tem como objetivo principal realizar a entrega do projeto.
- Nesta fase será envolvido, principalmente, o seguinte ator com suas respectivas responsabilidades:
 - o Analista de Negócio: terá responsabilidade de aprovar a entrega do produto realizando os testes de aceite.
- Ao final desta fase deve-se garantir a entrega do produto atendendo ao escopo e ao planejamento.

4 PROCESSOS DE ENGENHARIA DE SOFTWARE

Quando se fala em metodologia de desenvolvimento, deve-se pensar em institucionalisá-lo através de um processo de *software*. Um produto com qualidade só é possível se for construído de forma bem organizada, seguindo uma série de passos previsíveis através de um guia que ajude a controlar e chegar a um resultado de qualidade no prazo previsto.

A implantação de um processo pode ser visto como o conjunto de atividades, métodos, ferramentas, práticas e transformações que guiam pessoas na produção de *software* de forma mais assertiva e desenvolvimento ágil.

Um modelo de ciclo de vida organiza as macroatividades básicas, estabelecendo precedência e dependência entre elas. Um processo eficaz deve, claramente, considerar as relações entre as atividades, as pessoas que as executam (habilidades, treinamentos e motivação), os artefatos produzidos no desenvolvimento, as ferramentas e os procedimentos necessários.

No momento em que se decide construir um *software*, é fundamental também decidir qual processo será seguido. A criação de um projeto de *software* é uma atividade intelectual de alto nível de complexidade, necessitando melhor visibilidade na sua construção, sabendo-se de início quais são as etapas do projeto.

Aparentemente, todo e qualquer projeto de *software* pode ser construído sem que seja utilizado formalmente um guia predefinido para executá-lo. Porém, isso só é possível quando o projeto for construído como um passatempo ou *hobby* por uma ou duas pessoas em poucas horas de trabalho. Já no desenvolvimento profissional de *software*, o cenário pode ser radicalmente diferente e limitado, requeira muito retrabalho, tornando-o inviável caso não haja a utilização de um processo de *software* que reduza, dentro do possível, o desperdício de tempo e maximize os resultados, obtendo alta qualidade e satisfação em construí-lo.

Um processo de *software* é composto por métodos (aquilo que diz o que, em uma determinada tarefa), por ferramentas (que dão suporte automatizado aos métodos) e procedimentos (que fazem o elo de ligação entre os métodos e as ferramentas). Uma organização que possui um processo de engenharia de *software* deverá levar muito a sério estes três princípios de processo, a fim de que seus projetos de *software* sejam de sucesso, ou seja, equipes produtivas e *softwares* bem feitos.

Para facilitar as atividades dos analistas de processo nas organizações, geralmente o processo de Engenharia de *Software* é decomposto em diversos processos, tais como processo de gerência de projeto, processo de modelagem, processo de implementação, processo de qualidade, processo de gerência de configuração etc., que estão interligados e servem de apoio um ao outro durante o ciclo de vida do desenvolvimento do *software*.

Verifique, através da figura a seguir, que cada processo, por sua vez, é composto de atividades que também podem ser decompostas em pré-atividades,

subatividades, compostas por artefatos de entrada (insumos) e de saída (produtos) da atividade, os recursos necessários (humanos, *hardware*, *software* etc.) e os procedimentos (métodos, técnicas, modelos de documento etc.) a serem utilizados na sua realização.

FIGURA 14 – ELEMENTOS QUE COMPÕEM UM PROCESSO DE SOFTWARE

Processo de Software Processos Atividades Pré-atividades Subatividades Artefatos Insumos Produtos Recursos Recursos Humanos Ferramentas de Software Hardware Procedimentos Métodos Técnicas Roteiros	Exemplo: Atv1. Realizar levantamento de requisitos. <i>Pré-Atividade: Atv0</i> <i>Insumo: Documento Planejamento de Entrevista</i> <i>Produto: Documento Registro de Entrevista.</i> <i>Recurso Humano: Analista de Sistemas</i> <i>Procedimento: Técnica para Realização de Entrevistas</i> Atv2. Documentar requisitos. <i>Pré-Atividade: Atv1</i> <i>Insumo: Documento Registro de Entrevista.</i> <i>Produto: Especificação Textual de Requisitos.</i> <i>Recurso Humano: Analista de Sistemas</i> <i>Procedimento: Roteiro para Elaboração da Especificação Textual de Requisitos.</i>
---	--

FONTE: Disponível em: <<http://www.inf.ufes.br/~monalessa/PaginaMonalessa-NEMO/ES/Slides-1-ES-Introducao&ProcessoSoftware.pdf>>. Acesso em: 06 jul. 2015.

Para elaboração de um processo de *software* é importante que a organização possua uma boa ferramenta de documentação, que tenha inteligibilidade durante suas definições, permitindo que o progresso do processo seja visível externamente, tenha suporte a outras ferramentas CASE, com boa aceitabilidade e confiabilidade durante a utilização. Seja robusto para continuá-lo a despeito de problemas inesperados e tenha boa facilidade de manutenção e velocidade para construí-lo.

FIGURA 15 - ELABORAÇÃO DE PROCESSO DE SOFTWARE



FONTE: Disponível em: <http://www.projectbuilder.com.br/blog-pb/entry/dicas/5-motivos-para-documentar-seus-processos-e-adotar-uma-ferramenta-de-gp>. Acesso em: 26 jul 2015.

Para apoiar a definição de processos de *software*, diversas normas e modelos de qualidade foram propostos, dentre elas: ISO 9001, ISO/IEC 12207, ISO/IEC 15504, CMMI e MPS.Br - cujo objetivo é apontar características que um bom processo de *software* tem de apresentar, deixando a organização livre para estruturar essas características segundo sua própria cultura.

NOTA

Com o aumento da complexidade dos processos de *software*, passou a ser imprescindível o uso de ferramentas e ambientes de apoio à realização de suas atividades, visando, sobretudo, atingir níveis mais altos de qualidade e produtividade. Ferramentas CASE (*Computer Aided Software Engineering*) passaram, então, a ser utilizadas para apoiar a realização de atividades específicas, tais como planejamento e análise e especificação de requisitos.

IMPORTANTE

Diversas ferramentas CASE (*Computer Aided Software Engineering*) podem ser encontradas no mercado via *framework*, que trata sobre sua aplicação a fim de auxiliar no desenvolvimento de *software*. Durante este caderno de estudos você irá perceber alguns *frameworks* nas áreas de gerência de configuração, planejamento, codificação e testes de *software*.

Apesar dos benefícios do uso de ferramentas CASE individuais, atualmente, o número e a variedade de ferramentas têm crescido a tal ponto que levou os ES a pensarem não apenas em automatizar os seus processos, mas sim em trabalhar com diversas ferramentas que interajam entre si e forneçam suporte a todo ciclo de vida do desenvolvimento, que buscam combinar técnicas, métodos e ferramentas para apoiar o engenheiro de *software* na construção de produtos de *software*, abrangendo todas as atividades inerentes ao processo: gerência, desenvolvimento e controle da qualidade.

A escolha de um modelo de processo (ou modelo de ciclo de vida) é o ponto de partida para a definição de um processo de desenvolvimento de *software*.

Um modelo de ciclo de vida organiza as macroatividades básicas, estabelecendo precedência e dependência entre elas. Maiores detalhes sobre modelos de ciclo de vida de *software* serão apresentados no próximo tópico desta Unidade 1.

LEITURA COMPLEMENTAR

RESUMO

- A Engenharia de *Software* surgiu para auxiliar os profissionais de *software* nas crescentes dificuldades encontradas no processo de produção de soluções de *software*. A partir dos anos 60, a crise de *software* estourou devido ao fato de cada vez mais haver maior complexidade dos *softwares*, com alto índice de manutenibilidade, elevando o custo para mantê-lo.
- A Engenharia de *Software* foi fundada em 1969 por Fritz Bauer, a partir da necessidade de formalização e aperfeiçoamento dos métodos de construção de *software* para que seus projetos fossem entregues dentro de custo e prazo adequados.
- Ian Sommerville (2011) define a Engenharia de *Software* como uma disciplina da ES que se ocupa de todos os aspectos da produção de *software*, desde o início até o fim do desenvolvimento, contida por um conjunto de atividades parcialmente ou totalmente ordenadas para obter um *software* de qualidade.
- Roger Pressman entende que ES é composta por quatro camadas ou níveis: (1) foco na qualidade para obter satisfação dos clientes; (2) processos para definir as atividades, suas sequências e procedimentos; (3) métodos para definir as técnicas de execução das tarefas; (4) ferramentas para permitir a automatização dos processos e métodos de execução do projeto.
- Um processo de *software* não pode ser definido de forma universal. Para ser eficaz e conduzir a construção de produtos de boa qualidade, um processo deve ser adequado ao domínio da aplicação e ao projeto específico. Deste modo, processos devem ser definidos caso a caso, considerando-se as especificidades da aplicação, a tecnologia a ser adotada na sua construção, a organização onde o produto será desenvolvido e o grupo de desenvolvimento.
- A Engenharia de *Software* é composta pelos princípios de formalidade, abstração, decomposição, generalização, flexibilidade, padronização, rastreabilidade, desenvolvimento iterativo, arquiteturas baseadas em componentes, modelagem visual e verificação contínua de qualidade, cujo objetivo é implementar metodologias que definam a forma de desenvolvimento de *software* através de modelos, padrões, arquiteturas, métodos e processos que possam oferecer maior produtividade, eficiência, profissionalismo e criatividade das equipes envolvidas.
- A metodologia de desenvolvimento de *software* irá estabelecer produtos de trabalho padronizados que facilitem as atividades de manutenção de *software*, permitindo que os colaboradores sejam treinados, melhorando a qualidade

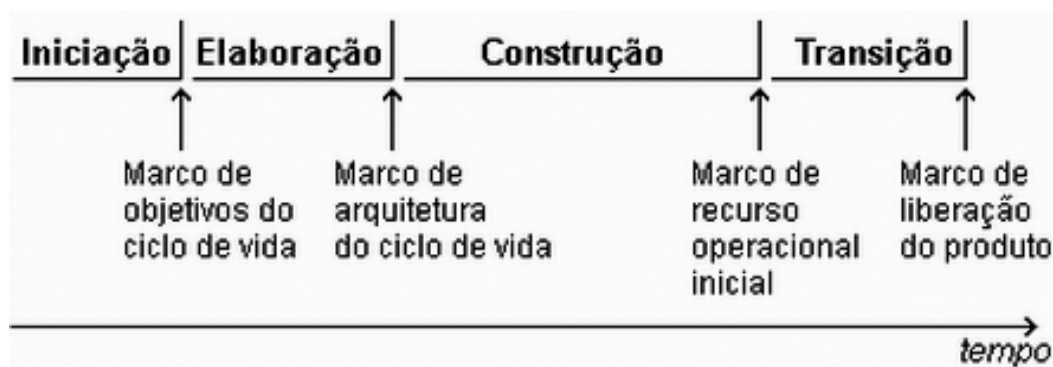
dos serviços através de um canal de comunicação uniforme entre os membros da equipe de desenvolvimento, para um maior aproveitamento dos seus recursos.

- Fernandes (1999) explica que, para uma boa metodologia de desenvolvimento de sistemas, é preciso haver consistência nos seguintes requisitos durante sua construção e utilização: padronização, flexibilidade, documentação, modularização e planejamento.
- A metodologia SWEBOK nos apresenta que a Engenharia de *Software* é composta por dez disciplinas, as quais compõem as áreas de conhecimento: (1) *Design de Software*, (2) *Construção de Software*, (3) *Teste de Software*, (4) *Manutenção de Software*, (5) *Gerência de Configuração de Software*, (6) *Gerenciamento de ES*, (7) *Engenharia de Processo de Software*, (8) *Ferramentas e Métodos de Software*, (9) *Qualidade de Software* e (10) *Requisito de Software*.
- Existem quatro fases de desenvolvimento de *software*, sendo que na fase de iniciação a ênfase é identificar o escopo do projeto; na fase de elaboração, a ênfase é análise e planejamento; na fase de construção, a ênfase é a codificação; e na fase de transição, a ênfase é a implantação.
- A construção de forma bem organizada de projeto de *software* só é possível através de um processo de *software* que pode ser visto como um conjunto de atividades, métodos, ferramentas que guiam as equipes na produção de *software* de produtividade e qualidade. Normalmente, um processo é composto de atividades, pré-atividades, subatividades, artefatos, recursos necessários e os procedimentos a serem utilizados na sua realização.
- A criação de um *software* é uma atividade intelectual de alto nível de complexidade, que exigirá o uso de processo para maior visibilidade do seu andamento e maior assertividade durante a tomada de decisão. Um processo é composto de métodos que relatam como fazer o projeto, de ferramentas que mostrem como automatizar os métodos e de procedimentos que documentem como a ferramenta executa os métodos, a fim de disseminar estas funções como guia padrão de desenvolvimento.
- As metodologias pesadas devem ser aplicadas apenas em situações em que os requisitos do *software* são estáveis e requisitos futuros são previsíveis. Estas situações são difíceis de serem atingidas, uma vez que os requisitos para o desenvolvimento de um *software* são mutáveis.

1 INTRODUÇÃO

Um ciclo de vida de desenvolvimento de *software* pode ser entendido como um roteiro de trabalho executado durante um projeto, no qual, em geral, cada atividade é constituída de macroetapas interdependentes, que fazem uso de métodos, técnicas, ferramentas e procedimentos para construção do produto de *software*, de forma adequada, seguindo critérios de qualidade e formando uma base sólida para o desenvolvimento.

FIGURA 16 - MACROETAPAS DO CICLO DE VIDA DE DESENVOLVIMENTO DE SOFTWARE



FONTE: Disponível em: <http://www.wthree.com/rup/process/workflow/manageme/images/co_phas1.gif>. Acesso em: 20 jul. 2015

O ciclo de vida de um *software* descreve as fases pelas quais o *software* passa desde a sua concepção (iniciação) até ficar sem uso algum (após transição), determinando os passos a serem seguidos no desenvolvimento de sistemas, mantendo uma padronização de trabalho e determinando as etapas de validação do projeto.

De maneira geral, as fases do ciclo de vida de um *software* são constituídas de planejamento, análise e especificação de requisitos, projeto, implementação, testes, entrega e implantação, operação e manutenção.

Uma vez estabelecido o escopo do projeto, se planeja o cronograma com as estimativas de recursos, custos e prazos do projeto. Durante a análise e

especificação, o escopo é refinado, relatando o que tem que ser feito para que na fase de projeto sejam modelados os requisitos tecnológicos. Identificando-se os principais componentes arquiteturais, cada unidade de *software* é implementada e, finalmente, o sistema como um todo deve ser testado.

Após o *software* ser homologado, deve ser entregue. Para tal, é necessário treinar os usuários, configurar o ambiente de produção e, muitas vezes, converter bases de dados para o processo de operação ser iniciado, quando o *software* passa a ser utilizado pelos usuários.

Por fim, após ter sido entregue para o usuário, sempre haverá a fase de manutenção do projeto, seja por identificação de erros, pelo *software* precisar ser adaptado para acomodar mudanças em seu ambiente externo, ou pela necessidade do cliente de funcionalidade adicional ou aumento de desempenho (FALBO, 2005).

2 MODELOS DE PROCESSO DE CICLO DE VIDA DE SOFTWARE

Os modelos de processo de ciclo de vida durante o desenvolvimento do *software* podem ser linear, incremental ou iterativo, logo, compreendê-los poderá auxiliar na adoção de um dos modelos mais adequados à realidade e necessidade da organização.

No modelo linear, o *software* é executado e entregue com todas as suas funcionalidades em apenas uma fase, mais simples de utilizar, porém vem sendo cada vez menos utilizado pelo fato de o tempo de entrega ser tipicamente longo. O modelo incremental realiza entregas de forma dependente, ou seja, uma versão básica é disponibilizada ao final do primeiro ciclo de desenvolvimento, e nos ciclos seguintes novas funcionalidades são agregadas, até que se tenha o produto completo. Já no modelo iterativo, uma versão básica de boa parte das funcionalidade é disponibilizada no primeiro ciclo e as funções melhoradas são disponibilizadas posteriormente.

Para cada uma destas categorias de ciclos de vida de *software* há um ou mais modelos formais disponíveis para adoção e os modelos mais conhecidos são:

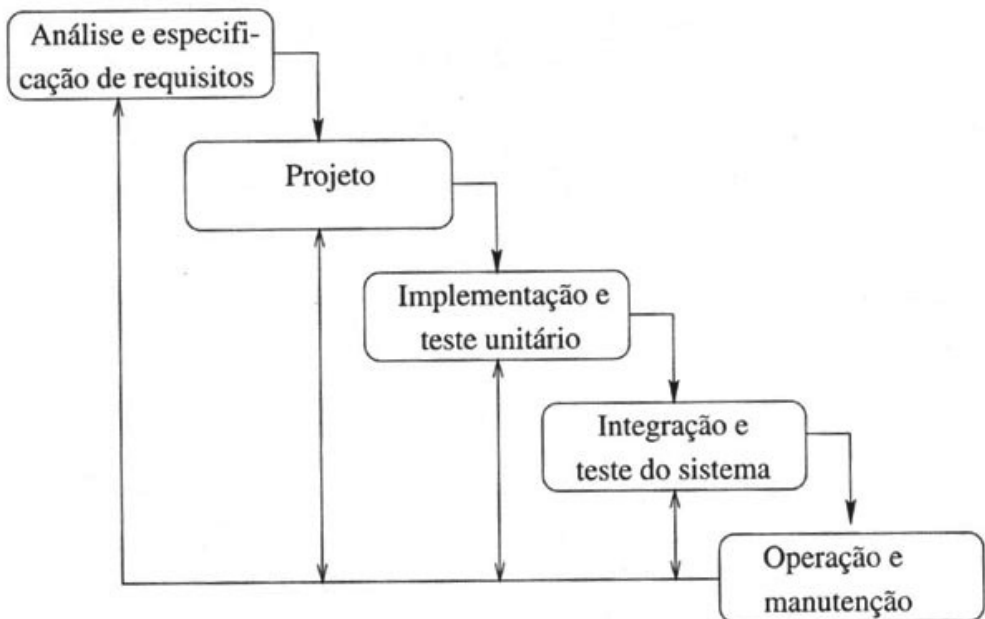
- O modelo Cascata ou sequencial.
- Modelo de Prototipação.
- Modelo Espiral.
- Modelo Iterativo e Incremental.
- Modelo Baseado em Componentes.
- Modelo em V.
- O Modelo RAD (*Rapid Application Development*).
- Modelo de Quarta Geração.

2.1 MODELO CASCATA OU SEQUENCIAL

O modelo cascata é um modelo de ciclo de vida clássico, pois trabalha numa abordagem sistemática em que as fases são estabelecidas pelas funções realizadas na engenharia convencional. Tem como característica marcante o fato de que todas as fases de desenvolvimento têm momentos de início e término bem definidos, sendo que uma fase só inicia se a anterior estiver concluída. Foi proposto por Winston W. Royce, no ano de 1970, e desde a década de 1980 é amplamente usado pela comunidade de desenvolvimento de *software*, pois é a mais conhecida, simples e fácil de trabalhar.

Conforme apresentado na Figura 17, ao final de cada fase ou marco de controle é feita uma revisão para avaliar se realmente pode-se avançar para a próxima e, quando apontar que o projeto não está apto a entrar na fase seguinte, ele permanece na fase corrente até que seja aprovado, ou seja, as atividades de especificação, codificação e testes seguem uma única disciplina rígida, com a qual nenhuma atividade se inicia sem que a anterior tenha sido encerrada e aprovada.

FIGURA 17 - CICLO DE VIDA CASCATA



FONTE: Disponível em: <<http://www.oficinadanet.com.br/artigo/gerencia/software-house-como-funciona-o-mercado-de-desenvolvimento>>. Acesso em: 18 jun. 2015

Embora o modelo cascata tenha fragilidades, ele é significativamente melhor do que uma abordagem casual de desenvolvimento de *software*, é considerado o modelo de fácil gestão, pelo fato de que suas etapas são bem definidas e sem sobreposição. É útil nos casos em que o domínio de aplicação é bem entendido e quando vários sistemas similares foram construídos anteriormente.

Como característica negativa deste modelo é a demora no atendimento devido aos retrabalhos tardios, ou seja, na maioria das vezes é praticamente impossível obter-se a totalidade de requisitos de maneira antecipada e em uma única etapa de um ciclo, por isso faz-se importante possibilitar o retorno para uma etapa sempre que for necessário. Como exemplo podemos citar quando os testes só ocorrerão no final de todo o processo, onde um erro sutil pode vir a exigir semanas de verificação para que se possa eliminá-lo, o que poderá impactar em atrasos no cronograma do projeto.

2.2 MODELOS POR PROTOTIPAÇÃO

As tecnologias de prototipação foram criadas no final dos anos 80 como alternativa a eliminar dificuldade de atender aos requisitos definidos pelos clientes e certificar-se de que efetivamente foram atendidas as suas necessidades.

A ideia deste modelo é produzir uma representação visual das funcionalidades que o *software* terá depois de pronto, permitindo avaliar as características antes que ele seja efetivamente desenvolvido e compreender com mais clareza como ficará sua usabilidade, trazendo maior velocidade de desenvolvimento e o envolvimento direto do usuário.

No modelo cascata se fazia um protótipo apenas como mecanismo para identificar ou auxiliar na identificação e elaboração dos requisitos finais do sistema e depois era descartado, mas por prototipação seu conceito tornou-se evolutivo, e o próprio protótipo passou a ser o produto entregue ao cliente.

Através deste modelo os desenvolvedores iniciam a especificação (requisitos) e codificação das partes essenciais do sistema em um protótipo, adicionando melhorias até se tornar o produto final. Sua aplicação permite a verificação da eficiência de determinadas rotinas ou fluxos de atividades, agilizando seu processo de desenvolvimento.

FIGURA 18 - MODELO DE PROTÓTIPO



FONTE: Disponível em: <<http://www.diegomacedo.com.br/modelos-de-ciclo-de-vida/?print=print>>. Acesso em: 19 jul. 2015

Importante destacar que este modelo é apropriado quando o cliente não obtém os requisitos de entradas e saídas devidamente definidos, quando o cliente participa ativamente do projeto, construção e validação do protótipo através de uma metodologia ágil de desenvolvimento, a fim de se criar uma interface bem próxima daquela que o *software* irá possuir.

Durante as etapas de avaliação e refinamento do protótipo há *feedback* constante a respeito do produto, aumentando a visibilidade do que está sendo desenvolvido, ajudando nos casos em que os analistas e os clientes não conhecem bem o domínio da aplicação e o cliente está apressado para receber o projeto com urgência.

IMPORTANTE

Para este tipo de ciclo de vida é preciso, antes, prevenir e orientar os clientes quanto à sua expectativa no prazo de entrega do projeto, pois acham que o protótipo já é o projeto pronto, e, neste modelo, se há muita dificuldade em se prever o tempo que será necessário para produzir um produto aceitável olhando somente para o protótipo. Sem levar em consideração requisitos de qualidade e sua manutenibilidade.

Existem diversas ferramentas para elaboração de protótipos de *software*, muitas linguagens de programação possuem a possibilidade de se fazer os protótipos do sistema, como, por exemplo, o Delphi e o Eclipse, porém outras, utilizadas pelos *designers*, como o Mockingbird, fazem os protótipos e depois só reaproveitam as telas para configurar e programar o restante do produto.

A utilização de prototipação é recomendada por diversos autores da área de Engenharia de *Software*, porém, a ideia básica deste modelo de processos de desenvolvimento de *software* é que o protótipo seja descartado e o processo construtivo siga as boas práticas de Engenharia de *Software*, evitando fragilidades da gestão de projeto e possa garantir semelhanças entre o protótipo e o *software* construído. Quando devidamente utilizado, reduzirá o retrabalho e atenderá rapidamente às expectativas dos usuários.

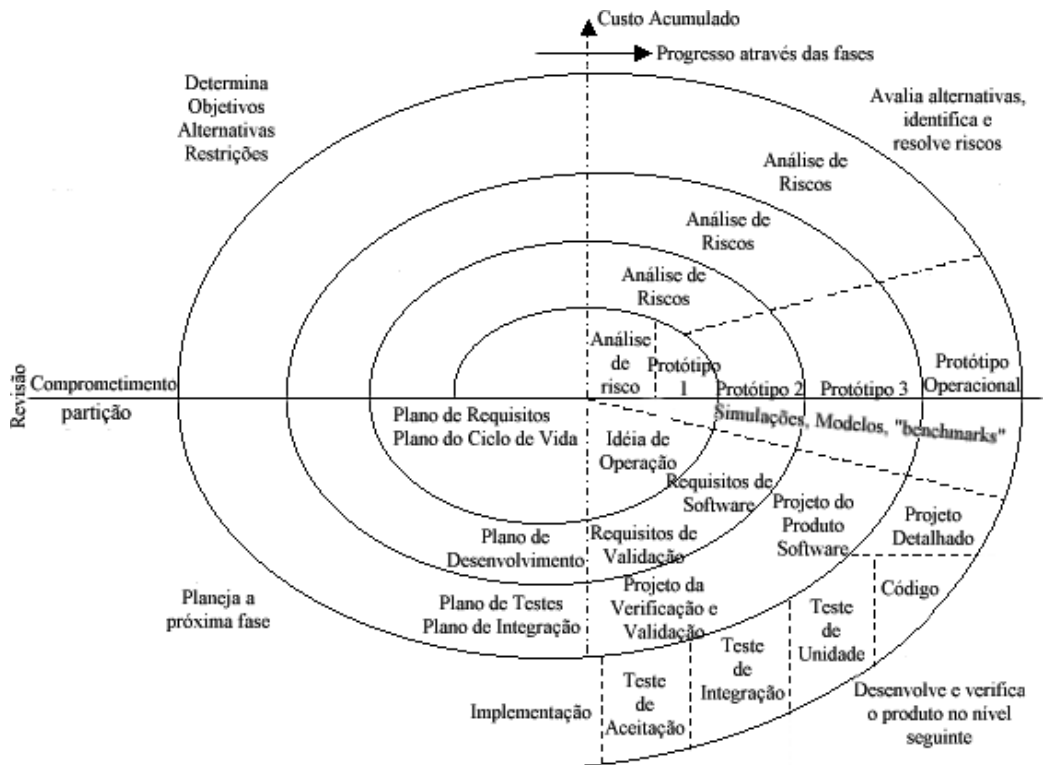
Um dos processos fundamentais durante o ciclo de vida de projeto de *software* é o levantamento de requisitos. Em muitos casos, as falhas estão na identificação correta das necessidades do cliente. No Tópico 4 desta Unidade 1 será apresentado sobre a área de Engenharia de Requisitos, disciplina extremamente importante para a engenharia de *software*.

2.3 MODELO ESPIRAL

O modelo espiral criado por Barry Boehm, no ano de 1988, traz uma abordagem das melhores características existentes nos modelos cascata e de prototipação vistos anteriormente, com ponto de vista orientado a riscos ao invés das abordagens orientadas à documentação e codificação.

Neste modelo, à medida que o desenvolvimento do *software* avança, percorre-se a espiral no sentido horário do centro para fora, incorporando inclusão de novos requisitos de forma evolutiva. Verifica-se que há sobreposição evolutiva durante quatro setores: primeiro, planejar os objetivos, as alternativas e restrições para tratamento dos riscos identificados; segundo, avaliação e redução de riscos, sua probabilidade e impacto durante cada fase do projeto; terceiro, com base nos riscos já identificados, desenvolve e valida estratégia de desenvolvimento para reduzir a probabilidade de ocorrência do risco; e quarto, realiza revisão e planejamento do projeto e decide pela continuidade ou não do próximo ciclo, seja durante as fases de elaboração dos requisitos, planos de implementação ou execução dos testes e integração do produto.

FIGURA 19 - MODELO ESPIRAL



FONTE: Disponível em: <<http://www.diegomacedo.com.br/modelos=-de-ciclo-de-vida/?print-print>>. Acesso em: 19 jul. 2015

Uma consideração importante para este modelo é que seu foco está em aumentar a qualidade do planejamento em cada ciclo, dando maior visibilidade para a gerência, especialmente na análise de riscos e favorecendo o desenvolvimento de versões incrementais do *software*. Como exemplo de riscos que representem algum impacto no resultado final do *software*, podemos citar problemas devido à falta de segurança e performance, atraso de entrega do projeto, incompatibilidade de ferramentas, entre outros.

Kechi Hirama (2011) explica que o processo Espiral é dividido em uma série de atividades, chamadas regiões de tarefas, nas quais são realizadas as seguintes atividades:

- Comunicação do Cliente, que visa estabelecer uma comunicação efetiva entre desenvolvimento e cliente.
- Planejamento, em que se definem recursos, cronograma e outras informações de projeto.
- Análise de Riscos, na qual se avaliam os riscos técnicos e gerenciais.
- Engenharia, que consiste na implementação de uma ou mais representações do *software*.

- Construção e Entrega, quando se implementa, testa, instala o *software* e se provê apoio ao usuário.
- Avaliação do Cliente, que objetiva obter realimentação do cliente baseada na avaliação das representações do *software* implementadas durante as atividades de engenharia e construção e entrega.

FONTE: Hirama (2011, p. 210)

Portanto, devido ao nível de gerenciamento ser mais complexo, vai exigir a necessidade de maior experiência da equipe de desenvolvimento, sobretudo os responsáveis pela gerência, que terão que ter maior experiência da equipe na análise de riscos e maior esforço para o desenvolvimento, aumentando consideravelmente os custos do projeto.

2.4 MODELO ITERATIVO E INCREMENTAL

É um modelo que tem uma abordagem que divide o desenvolvimento de *software* em ciclos, sendo que o desenvolvimento evolui em versões de novas funcionalidades até que o sistema completo esteja construído. Cada ciclo é constituído pelas fases de análise, projeto, implementação e testes, já discutidas no Tópico 2 desta unidade.

Foi criado em resposta às fraquezas do modelo cascata. Adequado quando é possível dividir os requisitos do sistema em partes gradativas do desenvolvimento, em que sua alocação é realizada em função do grau de importância atribuído a cada requisito, onde cada ciclo considera um subconjunto de requisitos que produz um novo incremento ou iteração do sistema que contém extensões e refinamentos sobre o incremento ou iteração anterior.

O modelo de ciclo de vida Iterativo apresentado na Figura 20, a seguir, corresponde em desenvolver ou melhorar o projeto de *software* em etapas diferentes através de iterações. Pode-se citar como exemplo: um projeto possui três módulos diferentes e que são independentes um do outro, os quais poderão, identificando e especificando requisitos em iteração diferentes do projeto, ou seja, desenvolver na iteração 1 o módulo 1, iteração 2 o módulo 3 e na iteração 3 o módulo 2.

Este tipo de modelo é utilizado, especialmente, quando se faz necessário buscar estratégias diferentes de se executar o planejamento do projeto, por exemplo, quando se deseja liberar uma funcionalidade mais importante primeiro, quando se precisa alocar profissionais específicos em determinado período do desenvolvimento do projeto ou, até mesmo, pelos recursos financeiros destinados a um determinado momento daquele projeto.

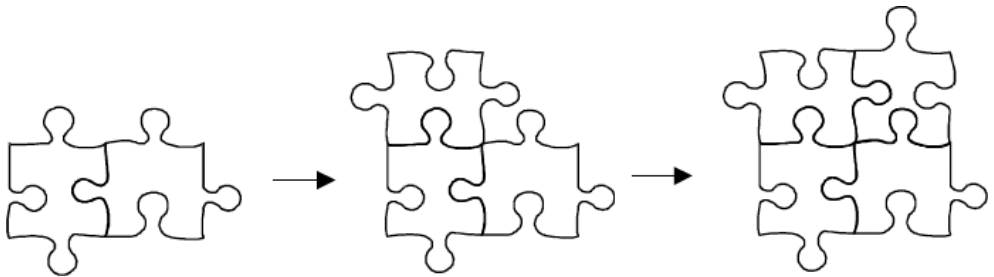
FIGURA 20 - MODELO DE CICLO DE VIDA ITERATIVO



FONTE: Disponível em: <http://wiki.sj.ifsc.edu.br/wiki/index.php/Ciclo_de_Vida_Iterativo_e_Incremental>. Acesso em: 19 jul. 2015

O modelo de ciclo de vida Incremental, apresentado na Figura 21, a seguir, demonstra que há dependência entre módulos de um sistema, portanto, para ser desenvolvido ou melhorado um segundo módulo do sistema, o primeiro módulo deverá estar 100% concluído e liberado para uso.

FIGURA 21 - MODELO DE CICLO DE VIDA INCREMENTAL



FONTE: Disponível em: <http://wiki.sj.ifsc.edu.br/wiki/index.php/Ciclo_de_Vida_Iterativo_e_Incremental>. Acesso em: 19 jul. 2015

Em cada ciclo ocorrem as atividades de análise de requisitos, projeto, implementação e teste, bem como a integração dos artefatos produzidos com os artefatos já existentes. Assim, o desenvolvimento evolui em versões, através da construção incremental e iterativa de novas funcionalidades, até que o sistema completo esteja construído.

Entre as principais vantagens dos modelos iterativo e incremental está a redução dos riscos envolvendo custos a um único incremento, pois, caso a equipe de desenvolvedores precisar repetir a iteração, a organização perde somente o esforço mal direcionado de uma iteração, não o valor de um produto inteiro.

O que conspira a favor deste tipo de modelo é a aceleração do tempo de desenvolvimento do projeto como um todo. Os desenvolvedores trabalham de maneira mais eficiente quando buscam resultados de escopo pequeno e claro, impedindo o risco de lançar o projeto no mercado fora da data planejada.

Outra evidência favorável a este tipo de ciclo de vida, que é muito comum hoje em dia nos projetos de *software*, é justamente que as necessidades dos usuários e os requisitos correspondentes não podem ser totalmente definidos no início do processo, precisando ser refinados em sucessivas iterações, possibilitando, com este modelo de operação, a facilidade de se adaptar a mudanças de requisitos.

Porém, o que conspira contra é que este modelo tem maior dificuldade de gerenciamento, pois as fases do ciclo ocorrem de forma simultânea, com amplo sincronismo entre atividades, pessoas, recursos etc., e também o cliente pode se entusiasmar excessivamente com a primeira versão do sistema e pensar que tal versão já corresponde ao sistema como um todo.

IMPORTANTE

A abordagem incremental e iterativa somente é possível se existir um mecanismo para dividir os requisitos do sistema em partes, para que cada parte seja alocada a um ciclo de desenvolvimento. Essa alocação é realizada em função do grau de importância atribuído a cada requisito.

2.5 MODELO BASEADO EM COMPONENTES

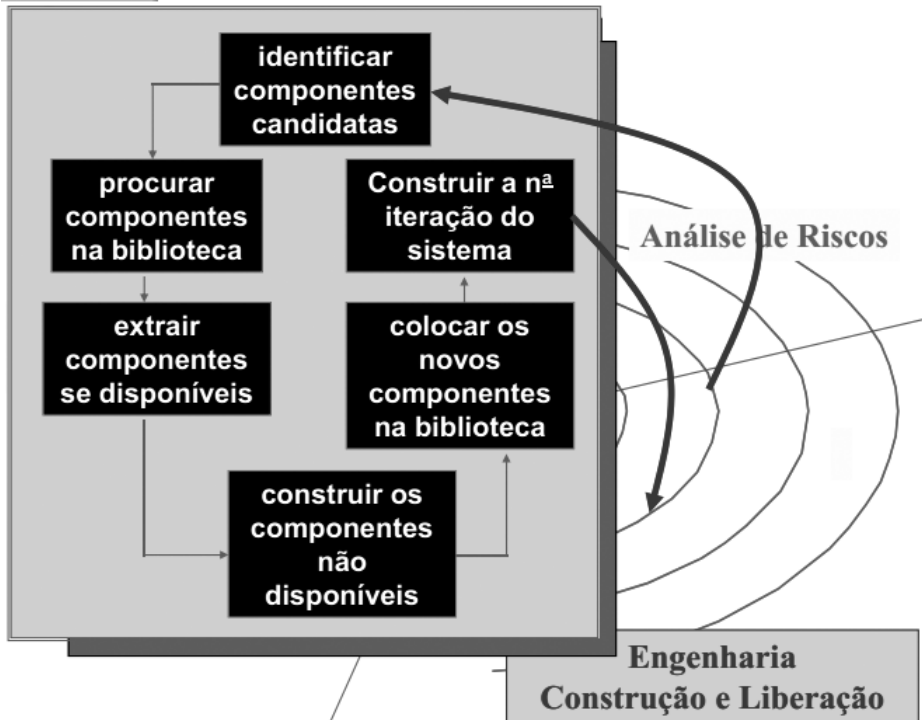
A ideia de que o *software* deveria ser componentizado surgiu na conferência patrocinada pelo Comitê de Ciência da Organização do Tratado do Atlântico Norte (OTAN), que lançou o termo Engenharia de *Software*, na Alemanha, em 1969, devido à necessidade de agrupar coerentemente rotinas relacionadas de forma a montar componentes que poderiam ser reutilizados em diversos sistemas.

O desenvolvimento baseado em componentes depende da existência de uma grande biblioteca disponível de componentes de *software* reusáveis utilizados pela empresa. É ideal que se tenha algum *framework* de integração bem padronizado desses componentes, facilitando o processo de reúso de *software*. Este modelo aproveita bem o conceito do desenvolvimento espiral através de reúso informal e independentemente do processo de desenvolvimento utilizado.

Seu processo de funcionamento é relativamente fácil: a partir do momento em que subir o requisito do cliente, é feita uma busca na biblioteca de componentes pelo componente que trata sobre este requisito para implementação, extraindo ele da biblioteca, fazendo a alteração (prática do reúso) e devolvendo este incremento na biblioteca deste *framework*; caso o componente já existir e estiver completo, faz-se o seu reúso. Porém, se não existir o componente candidato à alteração, deve-se criar um novo componente, desenvolvê-lo e fazer sua integração à biblioteca e liberar o produto pronto ao cliente. Esta reutilização possibilita a produção de programas a partir de partes já testadas e amplamente utilizadas.

IMPORTANTE

FIGURA 22 - CICLO BASEADO EM COMPONENTES



FONTE: Disponíveis em: <<http://metodologiasclassicas.blogspot.com.br/p/modelos-especializados-de-processos.html>>. Acesso em: 19 jul. 2015

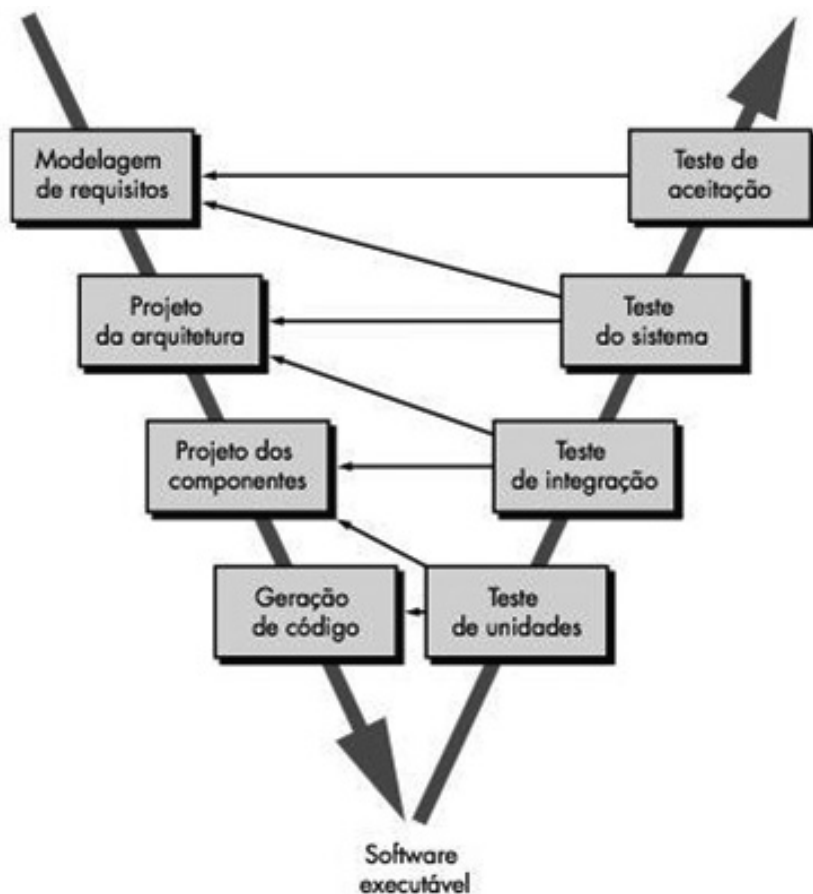
Dependendo da robustez tecnológica da biblioteca de componentes utilizada, os métodos de uma determinada classe podem ser reutilizáveis em diferentes aplicações e arquiteturas de sistema, oferecendo redução do tempo de desenvolvimento do custo do projeto. Estudos comprovam que esta redução de tempo e custo é bastante alta, já que o conceito de orientação a objetos permite o reúso de diferentes objetos, métodos dos componentes modelados.

Nota-se que as empresas que utilizam este modelo muitas vezes ficam presas pela tecnologia, não explorando novas soluções e impedindo, assim, atender às reais necessidades do usuário.

2.6 MODELO EM V

Este modelo enfatiza a estreita relação entre as atividades de teste com as demais atividades do processo de desenvolvimento de realizar a implementação do *software*. A análise e a especificação são realizadas com o apoio da área de testes, tomando como ponto de partida entender o problema e apresentar a melhor solução ao projeto. Parte-se da premissa de que, quanto mais tempo levar para entender, mais rápido se executa e menos erros são cometidos ao projeto.

FIGURA 23 - MODELO EM V



FONTE: Disponível em: <<http://www.devmedia.com.br/introducao-ao-modelo-casca-ta/29843>>. Acesso em: 19 jul. 2015

Neste modelo a ênfase está na decomposição de requisitos (lado esquerdo da figura) à integração dos testes (lado direito da figura). Os testes de unidade devem garantir que todos os aspectos detalhados na especificação do sistema foram codificados corretamente e, com os testes de integração, fazer a validação dos componentes utilizados. Quando a área da arquitetura passa a ser o foco, os testes de sistema validarão sua integração verificando se o sistema atende aos requisitos definidos na especificação técnica (projeto arquitetural) e, finalmente, os testes de aceitação pelo cliente buscam validar os requisitos para confirmar a homologação final do sistema.

Segundo Kechi Hirama (2011), as principais características do Modelo V são:

- Divisão entre atividades de desenvolvimento e de verificação e validação. As atividades de desenvolvimento são análogas às do processo cascata, porém a fase de testes é detalhada em testes de unidade, testes de integração, teste de validação e teste de sistema. Naturalmente, outros tipos de testes podem ser incluídos.
- Clareza nos objetivos de verificação e validação. Atividades de verificação significam responder à seguinte questão: “Estamos construindo certo o produto, ou seja, de maneira correta?”. No caso da validação a questão é: “Estamos construindo o produto certo, ou seja, o *software* é aquele acordado com o cliente?”. Os testes são instrumentos fundamentais para verificar e validar o *software* contra as suas especificações.
- Melhor planejamento dos testes. O fato de explicitar os tipos de testes e a sua correspondência com as atividades de desenvolvimento facilitam o planejamento ao especificar os testes e os recursos necessários.

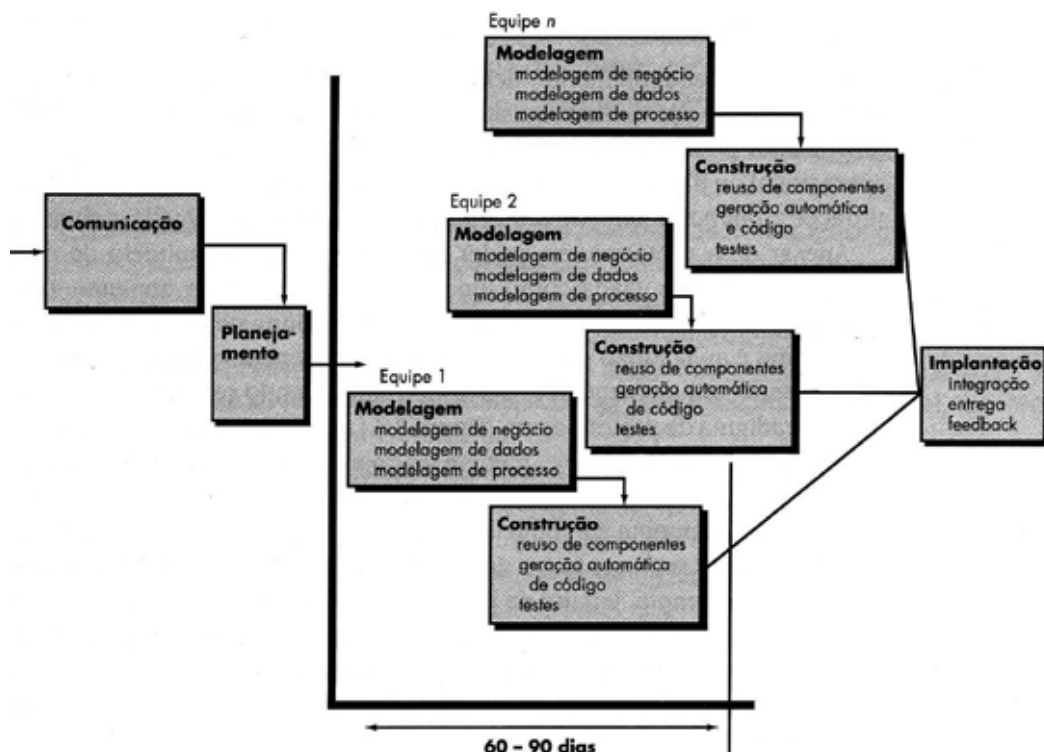
NOTA

2.7 MODELO RAD (*RAPID APPLICATION DEVELOPMENT*)

O modelo de desenvolvimento rápido de aplicações foi criado na década de 1990 por James Martin, a fim de propor um processo incremental mais curto de desenvolvimento entre as etapas de modelagem e codificação. É um processo que prima por metodologias ágeis, de forma que é recomendado realizá-lo em até 90 dias.

O planejamento é essencial, porque várias equipes trabalham em paralelo em diferentes funções do sistema. De forma genérica, ele possui as fases de comunicação, planejamento, diversos incrementos durante a modelagem e construção enfatizada pelo uso de componentes de *software*, conforme visto no modelo anterior, e concluído na implantação do sistema.

FIGURA 24 - CICLO DE VIDA RAD



FONTE: Disponível em: <<http://adsbaixarengenhariadesoftware.blogspot.com.br/2013/05/introducao-engenharia-de-software.html>>. Acesso em: 23 jul. 2015

O modelo RAD traz como vantagem a diminuição com custos em alterações, pois os requisitos incompletos podem ser vistos e completados durante o desenvolvimento, deixando o *software* mais próximo da necessidade do cliente, melhorando sua manutenção. Este modelo não é adequado para qualquer tipo de *software* ou necessidade de aplicação. Recomenda-se que a empresa possua uma plataforma de desenvolvimento que privilegie a agilidade, ideal para ambientes que possam fazer uso de outros modelos ágeis, como no caso do desenvolvimento baseado em componentes ou classes preexistentes, como APIs (*Application Programming Interface* ou Interface de Programação de Aplicativos).

2.8 MODELO DE QUARTA GERAÇÃO

O modelo de quarta geração é baseado na utilização de ferramentas de alto nível próxima à linguagem natural para especificação e desenvolvimento de *softwares*. Há uma mudança de paradigmas da prototipação na forma do desenvolvimento de *software*, otimizando sua mão de obra. Pode abranger as ferramentas com possível facilidade ao gerenciamento de bancos de dados, geração em diferentes visões de relatórios, prototipação de interfaces, ferramentas de linguagem de programação inteligentes na geração de códigos, utilizando

recursos gráficos de alto nível e recursos da inteligência artificial explorados nos conceitos ontológicos e semânticos das aplicações.

Em geral, seu uso é limitado ao desenvolvimento de um universo restrito de sistemas, permitindo o desenvolvimento de produtos para domínio de aplicações muito específicos, pois se restringe pela dificuldade de utilização das ferramentas, ineficiência do código gerado e manutenibilidade questionável. Além disso, ainda não há ambientes que possam ser utilizados com igual facilidade para todas as categorias de *software*.

IMPORTANTE

Além dos modelos de ciclo de vida descritos, encontramos na literatura muitos outros modelos, e cada um deles, geralmente, enfoca um determinado tipo de projeto específico, ou seja, depende do contexto do projeto onde ele será aplicado. Nenhum modelo de ciclo de vida é tido como ideal, pois aquele que é mais rápido e eficiente em certos projetos pode ser mais lento ou complexo quando o domínio do problema é trocado.

O mais importante é que as empresas de desenvolvimento de *software* tenham algum modelo de ciclo de vida, ou melhor, um “catálogo” de modelos personalizado de acordo com a sua estrutura organizacional e projetos/demandas.

- Um ciclo de vida de desenvolvimento de *software* define o roteiro de trabalho do projeto, que é composto por fase, atividades e suas tarefas, utilizando métodos, técnicas, ferramentas e procedimentos na construção do produto de *software*.
- As fases do ciclo de vida são constituídas de planejamento (iniciação), análise e especificação de requisitos, projeto (elaboração), implementação (construção), testes, entrega e implantação (transição) e, por fim, operação e manutenção, passando desde a sua concepção até ficar sem uso algum após o término do projeto.
- Os modelos de processo de ciclo de vida de *software* podem ser linear, incremental ou iterativo, logo, compreendê-los poderá auxiliar na adoção de um dos modelos mais adequados à realidade e necessidade da organização.
- Modelo cascata ou sequencial possui um ciclo de vida clássico, em que todas as fases de desenvolvimento possuem início e fim bem definidos e não avança sem estiver com a fase anterior 100% concluída.
- O objetivo das tecnologias de prototipação é produzir uma representação visual das funcionalidades que o sistema terá depois de pronto, permitindo ao usuário e à equipe de desenvolvimento avaliar as características antes que ele seja efetivamente implementado e entregue.
- Inicialmente, o modelo de prototipação desenvolve uma visão da sua interface e depois reaproveita as telas para configurar e programar o restante do produto de *software*, trazendo maior rapidez na construção do projeto e aceitação dos usuários.
- O modelo espiral traz uma abordagem orientada à gestão de riscos ao invés de apenas orientar a documentação e codificação, como nos casos do modelo cascata e de prototipação.
- Também no modelo espiral, à medida que o projeto de *software* avança são incorporados novos requisitos de forma evolutiva, com sobreposição de atividades em cada fase do projeto, aumentando a qualidade do planejamento em cada ciclo e dando maior visibilidade à gerência.
- O modelo iterativo e incremental permite dividir o escopo do projeto em partes gradativas do desenvolvimento, evoluindo o projeto em versões de novas funcionalidades até o sistema estar completo.

- O modelo iterativo pode desenvolver os módulos ou funcionalidade de forma independente e liberar o projeto em partes; já no modelo incremental, poderá desenvolver em módulos, porém há dependência de funcionalidades entre estes módulos para um próximo módulo ser implementado, permitindo a liberação após o acréscimo de funcionalidades feitas nas iterações anteriores.
- O modelo baseado em componentes permite agrupar rotinas relacionadas de forma a montar componentes que podem ser reutilizados em diversos módulos do sistema.
- A partir do momento em que sobe o requisito do cliente, é feita uma busca na biblioteca de componentes, que trata sobre este requisito, e se já existir e estiver completo, faz-se o seu reuso; caso contrário, criar um novo componente e fazer sua integração à biblioteca, liberando o produto pronto ao cliente.
- O foco do modelo em V dá ênfase nas atividades de testes durante a análise, implementação e homologação do sistema, primeiramente validando-os antes do desenvolvimento, garantindo maior entendimento do problema e evitando enganos e retrabalhos.
- Modelo RAD (*Rapid Application Development*) propõe um ciclo de vida rápido de desenvolvimento utilizando um processo incremental entre as etapas de modelagem e codificação durante um período de até 90 dias.
- Já o modelo de quarta geração trata dos modelos de última geração, utilizando ferramentas de alto nível através da inteligência computacional, explorando os paradigmas da ontologia e semântica das aplicações, ou seja, próxima à linguagem natural.

REQUISITO DE SOFTWARE

1 INTRODUÇÃO

A elaboração de um projeto, seja ele para criar um produto novo ou melhorá-lo, surge tipicamente em função de algum problema, oportunidade ou necessidade de negócio. A partir do momento em que for decidido iniciar um processo de construção de um *software*, deve-se definir o escopo do projeto através de uma lista de funcionalidades que se deseja disponibilizar para seus usuários no sistema, no qual estas necessidades identificadas são denominadas de requisito. Portanto, requisito é uma definição formal e detalhada de uma função do sistema.

Machado (2011) define requisitos como um conjunto de condições ou capacidades necessárias que um *software* deve possuir para que o usuário possa resolver um problema ou atingir um objetivo, ou para atender as necessidades ou restrições da organização ou dos outros componentes do sistema.

A análise e especificação dos requisitos têm vital importância no desenvolvimento de *softwares*, pois é nesta etapa da engenharia de *software* que são levantadas as informações de grande importância para a construção adequada do sistema (considerado marco decisivo de sucesso ou fracasso dos projetos).

A extração de requisitos é o processo de transformar o conhecimento tácito, que está na mente dos usuários, em conhecimento explícito via documentação formal. Essa transformação só é possível através da determinação dos objetivos do produto e das restrições para a sua operacionalidade, através de uma análise do problema e documentação dos resultados. A saída do processo de extração de requisitos é um documento de especificação do sistema que deve dizer o que o produto a ser desenvolvido deverá fazer, e não como deve ser feito.

IMPORTANTE

Embora seja óbvio, é importante que se chame a atenção para o fato de que os requisitos são a base sobre a qual serão apoiadas todas as demais atividades do processo de *software*. Se os requisitos não forem devidamente especificados, o sucesso do projeto poderá estar comprometido desde seu início.

Estudos realizados por Arteiro (2015) mostram porque trabalhar com requisitos de *software* é tão importante. De acordo com um estudo do *Standish Group*, no ano de 2014, cinco dos oito principais fatores de falhas em projetos estão relacionados a requisitos (Tabela 1). Estes são: requisitos incompletos, baixo envolvimento do cliente, expectativas não realistas, mudanças nos requisitos e requisitos desnecessários.

TABELA 1 - PRINCIPAIS CAUSAS DE FALHAS EM PROJETOS DE SOFTWARE

Problema	%
Requisitos incompletos	13.1
Baixo envolvimento do cliente	10.6
Falta de recursos	12.4
Expectativa não realista	9.9
Falta de suporte gerencial	9.3
Mudanças nos requisitos	8.7
Falta de planejamento	8.1
Requisitos desnecessários	7.5

FONTE: Disponível em: <http://www.cin.ufpe.br/~in1020/arquivos/monografias/2013_2/iveruska.pdf>. Acesso em: 24 jul. 2015

Como o requisito é uma documentação que diz o que o *software* deverá realizar, o sucesso do projeto depende de sua definição clara, e a engenharia de requisitos é considerada uma etapa do processo de *software* muito importante, pois é extremamente árduo quando se elabora requisitos de *software* de forma vaga e incompleta, gerando retrabalho para a equipe do projeto.

Simpoi publicou em 2008 que 70% dos problemas encontrados em *softwares* são de especificação, ou da falta dela, sendo que apenas 60% das informações formais e informais estão documentadas. Uma das principais razões para isso

encontra-se no perfil dos desenvolvedores, de modo geral, definido por pessoas talentosas, que não têm interesse em seguir disciplinas ou normas, consideram seu trabalho como algo artístico e criativo, ao invés de algo sujeito a procedimentos de engenharia de requisitos.

Carvalho e Chiossi (2001) apontam algumas dificuldades no processo de obtenção de requisitos:

- falta de conhecimento do usuário das suas reais necessidades e do que o produto de *software* pode lhe oferecer;
- falta de conhecimento do desenvolvedor do domínio do problema;
- domínio do processo de extração de requisitos pelos desenvolvedores de *software*;
- comunicação inadequada entre desenvolvedores e usuários;
- dificuldade do usuário em tomar decisões;
- problemas de comportamento;
- questões técnicas.

FONTE: Carvalho e Chiossi (2001, p. 148)

Os requisitos de negócio são declarações, em uma linguagem natural, como diagramas de quais serviços são esperados do sistema e as restrições sob as quais ele deve operar; os requisitos de sistemas seriam os detalhamentos, as funções, os serviços e as restrições operacionais do sistema. O documento de requisitos de sistema (chamado de especificação funcional) necessita ser preciso e validado entre o cliente e o desenvolvedor. Este documento deve dizer exatamente como dever ser implementado.

Numa definição mais abrangente e não relacionada especificamente a alguma área de conhecimento, um requisito pode ser considerado como uma condição indispensável para alcançar determinado resultado. Essa definição, puramente linguística, ajuda a direcionar uma linha de raciocínio que justifica a importância dos requisitos nas atividades de criação de *softwares* (GIANESINI, 2008, p. 22).

De acordo com Lopes (2004), um requisito é uma característica do *software* necessária para o usuário solucionar um problema de forma a atingir um objetivo. Uma característica de *software* que deve ser realizada ou implementada por um sistema ou componente de sistema para satisfazer um contrato padrão, especificação ou outra documentação formalmente imposta.

Diversas atividades de requisitos de *software* ocorrem ao longo de todo o ciclo de vida do *software*, um trabalho que consiste na análise de requisitos para identificar, quantificar, definir, especificar, documentar, rastrear, priorizar e classificar os principais problemas que o futuro *software* deve resolver. A seguir serão apresentados os três tipos de requisitos de *software*.

1.1 REQUISITOS FUNCIONAIS

Para Sommerville (2011), os requisitos funcionais são declarações de serviços que o sistema deve fornecer, de como o sistema deve reagir a entradas específicas e de como o sistema deve se comportar em determinadas situações. Em alguns casos, os requisitos funcionais também podem explicitar o que o sistema não deve fazer.

Já segundo Martins (2010), os requisitos funcionais são aqueles que definem o comportamento do sistema, capturados por meio de casos de uso, que documentam as entradas, os processos e as saídas geradas.

1.2 REQUISITOS NÃO FUNCIONAIS

Sommerville (2011) explica que os requisitos não funcionais são restrições ao serviço ou funções disponíveis no sistema. Incluem restrições tecnológicas no processo de desenvolvimento e restrições impostas pelas normas. Ao contrário das características individuais ou serviços do sistema, os requisitos não funcionais, muitas vezes, aplicam-se ao sistema como um todo.

Martins (2010) define que os requisitos não funcionais são compostos por características não necessariamente associadas ao comportamento, com o objetivo de definir características do sistema conforme observadas pelo cliente, apontando o desenvolvimento na direção correta.

Sommerville (2011) menciona também que, na realidade, a distinção entre diferentes tipos de requisitos não é tão clara como sugerem estas definições simples. Um requisito de usuário relacionado com a proteção, tal como uma declaração de limitações de acesso a usuários autorizados, pode parecer um requisito não funcional. No entanto, quando desenvolvido em mais detalhes, esse requisito pode gerar outros requisitos, claramente funcionais, como necessidade de incluir recursos de autenticação de usuários no sistema.

A norma ISO/IEC 9126 (2015) caracteriza os requisitos não funcionais referentes à qualidade dos produtos usando os seguintes conjuntos:

- Funcionalidade: Adequação; Acurácia; Interoperabilidade; Segurança de acesso.
- Confiabilidade: Maturidade; Tolerância a falhas; Recuperabilidade.
- Usabilidade: Inteligibilidade; Apreensibilidade; Operacionalidade; Atratividade.
- Eficiência: Comportamento em relação ao tempo; Comportamento em relação aos recursos.
- Manutenibilidade: Analisabilidade; Modificabilidade; Estabilidade; Testabilidade.
- Portabilidade: Adaptabilidade; Capacidade para ser instalado; Coexistência; Capacidade de ser substituído.

1.3 REQUISITOS INVERSOS

Segundo Pinto (2012), estes são requisitos que o foco do projeto não contemplará. Reconhece-se a existência destes requisitos, mas será declarado

de forma explícita que o projeto não irá abordá-lo ou não entregará aqueles requisitos, sendo de vital importância para não causar expectativas por parte do cliente ou usuários, delimitando o escopo do projeto.

Gomede (2011) comenta que estes requisitos definem estados e situações que nunca podem acontecer.

Pode-se dizer que os elementos da Engenharia de *Software* constituem o foco da gestão de requisitos. Não há a necessidade de que a equipe tenha experiência em gestão de requisitos, mas, se toda empresa que presta serviços de Engenharia de *Software* tiver uma célula de trabalho especializada em gestão de requisitos para dar suporte às áreas de interesse, sempre terá um ponto de partida para o correto planejamento do projeto e, tendo estes itens corretamente descritos e negociados, quase não existirá o risco de o sistema iniciar de forma errada e imprecisa.

Gerenciamento de requisitos é uma das atividades fundamentais ao processo de desenvolvimento de *software*. Constitui a base para a definição da arquitetura do sistema, para a implementação propriamente dita, para geração dos casos de testes e para validação do sistema junto ao usuário. Gerenciamento de requisitos está relacionado ao processo de controlar todo o processo de desenvolvimento tendo como referência a *baseline* de requisitos. Este processo visa manter planos, artefatos e atividades de desenvolvimento consistentes com o conjunto de requisitos definidos para o *software* (SAYÃO; BREITMAN, 2009, p. 1).

2 TÉCNICAS DE LEVANTAMENTO DE REQUISITOS

Observa-se que a maioria dos problemas no levantamento de requisitos está relacionada a não utilizar uma técnica adequada para extrair os requisitos do sistema, e o analista não os descreve de modo claro, conciso e consistente com todos os aspectos significativos do sistema proposto, pelo fato de, muitas vezes, o usuário principal do sistema não saber o que quer que o sistema faça ou sabe e não consegue transmitir para o analista.

A utilização de técnicas de levantamento de requisitos é recomendada por diversos autores de engenharia de requisitos e personalidades da área Engenharia de *Software*. Algumas das principais técnicas serão apresentadas a seguir, de forma resumida:

- *Brainstorming*: consiste em uma “tempestade de ideias”, sem julgamentos ou análises, que ocorre em um ambiente descontraído e informal. É ideal para buscar ideias de novos produtos.
- JAD: técnica utilizada para promover cooperação, entendimento e trabalho em grupo entre usuários e desenvolvedores.
- Análise de documentos quantitativos: consiste na análise de documentos, como formulários e relatórios.

- Reunião: é utilizada para licitação de requisitos em grupo, com o objetivo de promover o intercâmbio de ideias, sugestões e opiniões entre os participantes, visando à aceitação de um ponto de vista.
- Prototipagem: utilizado para atrair aspectos críticos quando se tem domínio mínimo da aplicação.
- Entrevista: conversa com o usuário para extrair tópicos importantes.
- Questionários: extrair respostas através de questões subjetivas e objetivas.
- Observação: observar o comportamento e o ambiente do indivíduo que toma decisões pode ser uma forma bastante eficaz de levantar informações que, tipicamente, passam despercebidas usando outras técnicas.
- Levantamento Orientado a Ponto de Vista: tem como objetivo captar pontos de vista dos usuários, analisar as diferenças e similaridades, formando, assim, o requisito do sistema.
- Etnografia: forma utilizada para entender a organização, sua cultura e o objetivo que o sistema deve alcançar.
- Caso de Uso: especifica um comportamento externo de um sistema descrevendo um conjunto de sequências de ações realizadas pelo sistema para produzir um resultado de valor observável por um ator, realizado através de uma interação típica entre um ator (usuário, outro sistema computacional ou um dispositivo) e um sistema.

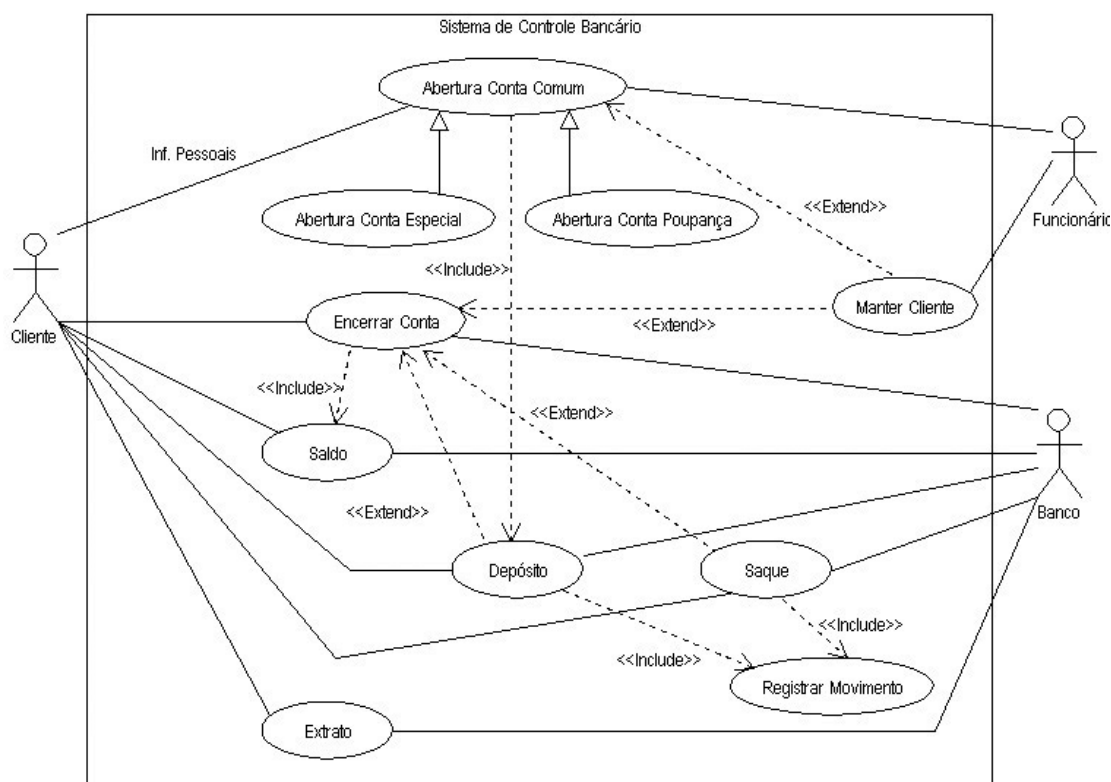
IMPORTANTE

Todas as técnicas de levantamento de requisitos possuem vantagens e desvantagens a serem consideradas devido às inúmeras complexidades dos projetos, porém, a utilização de mais de uma técnica, de forma combinada, irá ajudar a melhorar a qualidade da completude dos requisitos.

Quando o cliente ou usuário interno ou externo de um projeto de *software* contrata a área de tecnologia, é preciso definir um bom atendimento destas necessidades ou desejos, então a área de tecnologia precisa elaborar, de uma forma clara, precisa e consistente, um documento com as definições mais detalhadas do sistema para que o cliente possa compreender e validar os requisitos.

Na fase de análise de requisitos, muitas organizações utilizam a modelagem para contextualizar um projeto. Para deixar mais clara esta ideia, os exemplos a seguir detalham requisitos de um “Sistema de Controle Bancário” extraídos do livro UML2. A figura detalha o diagrama de caso de uso que permite uma visão dos requisitos funcionais do sistema de forma rápida e objetiva.

FIGURA 25 - DIAGRAMA DE CASO DE USO



FONTE: Guedes (2011)

O quadro a seguir apresenta a documentação do requisito do caso de uso “abrir conta especial”, demonstrando forte relação do usuário com a funcionalidade do sistema. É recomendado que seja criada uma relação forte entre ambos, para que seja fácil de localizá-los e atualizá-los em caso de necessidade.

QUADRO 1 – DOCUMENTAÇÃO TEXTUAL DO CASO DE USO

Nome do CDU:	Abrir Conta Especial
CDU Geral:	Abrir Conta
Ator:	Funcionário
Resumo:	Este Caso de Uso descreve as etapas necessárias para a abertura de uma Conta Especial para um Cliente.
Pré-Condições:	O pedido de abertura deve ser aprovado.
Pós-condições:	É necessário realizar um depósito inicial.
Fluxo:	Passo 1: O funcionário solicita a abertura de Conta Especial. Passo 2: O funcionário consulta o cliente por seu CPF ou CNPJ. Passo 3: É definido o valor limite do cheque especial. Passo 4: É inserida uma senha de acesso. Passo 5: A conta é criada. Passo 6: É fornecido o valor a ser depositado. Passo 7: É realizado o registro do depósito. Passo 8: É emitido o cartão da conta.
Restrições / Validações:	Restrição 1: Para abrir uma conta especial é preciso ser maior de idade. Restrição 2: É necessário estar empregado e o salário ser superior a 500,00. Restrição 3: O valor mínimo de depósito inicial é R\$ 50,00.

FONTE: Guedes (2011)

